软件体系结构课程电子教材

从需求、架构到质量保障

适用对象：软件工程 / 计算机相关专业课程复习

版本：学生阅读版 v5.1

课程范围：第 1 章至第 8 章

前言与使用说明

前言

本教材依据《软件体系结构》课程讲授内容整理，面向软件工程、计算机及电子信息相关专业学生的课程复习、项目实践和期末考试准备。它不是传统意义上的完整理论教材，而是一份围绕课堂主线、课程项目和工程判断能力组织的学习资料。

使用本教材时，建议结合每次课 PPT、课后作业和课程项目材料。PPT 用于把握课堂讲授结构，课后作业用于训练方法应用，课程项目用于把需求、建模、架构、测试和交付材料串成完整工程闭环。

本教材的核心目标是帮助学生回答一个问题：当一个软件系统从“功能能跑”进入“多人协作、长期演进、真实运行”的工程环境时，应该如何识别复杂性、组织开发过程、表达系统结构、做出架构决策，并通过测试和交付验证这些决策。

如何使用本教材

使用场景	建议方法
课前预习	先阅读本章导读、学习目标和核心概念，了解课堂将讨论的问题。
课后复习	重点阅读方法框架、贯穿案例、常见误区和本章小结。
项目材料准备	对照“项目提示”和章末项目实践题，把本章方法转化为课程项目产出。
期末复习	使用“考试提示”、复习题、附录 A 术语表和附录 B 题目索引进行集中复习。

课程知识地图

章节	主题	解决的核心问题
第 1 章	复杂性与软件工程全景	为什么会写代码不等于能控制复杂系统
第 2 章	过程模型与开发节奏	面对不同复杂性，应选择怎样的开发节奏
第 3 章	项目管理与风险控制	如何把目标、范围、计划和风险变成可执行动作
第 4 章	需求工程	如何把用户声音转化为可验证规格
第 5 章	架构表达与系统建模	如何用图和模型把系统讲清楚
第 6 章	架构设计与架构风格	如何做出有理由、可审查的架构决策
第 7 章	测试与质量保障	如何验证架构和质量目标是否成立
第 8 章	发布、运维与可观测性	如何上线、回滚、观察与复盘

课程项目产出路径

项目里程碑	主要产出	主要关联章节
M1：选题与需求	选题说明、利益相关者、典型用户、核心用例、需求规格初稿	第 1 章、第 2 章、第 4 章
M2：建模与结构表达	C4 图、用例图、流程图、核心类图/数据模型、建模检查清单	第 4 章、第 5 章
M3：架构决策与 ADR	架构风格说明、模块划分、关键质量属性、ADR、技术债记录	第 6 章
M4：测试、交付与运维说明	测试方案、质量门禁、部署说明、回滚方案、日志与可观测性说明	第 7 章、第 8 章

版本说明

当前版本：学生阅读版 v5.1

目录

前言与使用说明 2

第 1 章 从能写代码到能控复杂系统 5

第 2 章 软件过程模型与开发节奏选择 11

第 3 章 项目管理、可行性分析与风险控制 17

第 4 章 需求工程：从用户声音到可验证规格 25

第 5 章 架构表达与系统建模 32

第 6 章 架构设计、架构风格与分布式系统 40

第 7 章 软件测试、质量保障与安全验证 62

第 8 章 发布、交付、运维与可观测性 69

附录 A 核心术语速查表 76

附录 B 期末复习题索引 78

附录 C 课程项目产出模板索引 80

第 1 章 从能写代码到能控复杂系统

1.1 本章导读：为什么会写代码不等于能控复杂系统

软件系统最容易让初学者产生误解的地方在于：一个功能能运行，常常被误认为一个系统已经完成。对个人练习而言，这种判断有时可以成立；对多人协作、长期运行、涉及真实用户的数据系统而言，它远远不够。

以“实验室预约与设备管理系统”为例，最初的功能看起来并不复杂：学生提交预约，教师或实验员审批，系统记录设备状态，并发送通知。若只从代码实现角度看，可以很快写出页面、接口和数据库表。但系统一旦进入真实环境，就会遇到新的问题：两个学生同时预约同一台设备怎么办？通知服务失败是否影响预约成功？教师、实验员、学生看到的数据边界是否一致？学院以后要接入门禁、统一认证和设备台账，原来的代码是否还能扩展？

这些问题说明，软件工程关心的不只是“把功能写出来”，而是把功能放进一个可协作、可演进、可验证、可交付的工程系统中。软件体系结构课程的起点，也正是从“写代码”转向“控制复杂系统”。

【你要记住】功能能跑只是系统进入工程状态的最低条件。工程上真正要判断的是：系统是否能在变化、约束、协作和故障中保持可控。

1.2 一个反面案例：能跑但不可控的预约小系统

某项目组在两周内完成了实验室预约系统原型。学生可以选择设备和时间段，提交后数据库写入一条预约记录；教师可以在后台审批；预约成功后系统发送邮件。演示时一切顺利，页面也比较完整。

问题出现在试运行阶段。第一天，两个学生几乎同时预约同一台设备，系统生成了两条有效预约。第二天，邮件服务不稳定，提交预约的接口一直等待邮件发送结果，导致学生页面长时间转圈。第三天，学生通过修改浏览器请求参数访问了本不该看到的审批接口。第四天，学院提出后续要统计设备利用率，项目组发现原来的预约状态字段含义混乱，无法直接生成报表。

这个系统不是没有代码，也不是没有功能，而是没有把复杂性当成设计对象。它把“能提交表单”误认为“能支撑预约业务”，把“页面隐藏按钮”误认为“权限控制”，把“同步调用邮件接口”误认为“通知能力”，把“数据库能存”误认为“状态可分析”。

改进方向并不一定是使用更复杂的框架，而是先回答四个问题：需求会不会变化？质量属性是否明确？团队如何协作？运行和交付有什么约束？这四类问题构成了本课程理解复杂系统的基本入口。

1.3 软件复杂性的四个来源

软件复杂性通常不是单一来源造成的，而是多种力量叠加的结果。

第一类是需求规模与变化速度。实验室预约系统初版只做“预约”，后续可能增加设备借用、耗材管理、实验安排、门禁联动、学院统计。需求从一个表单扩展成一组业务流程，原来写死在页面或接口中的逻辑就会变成阻力。

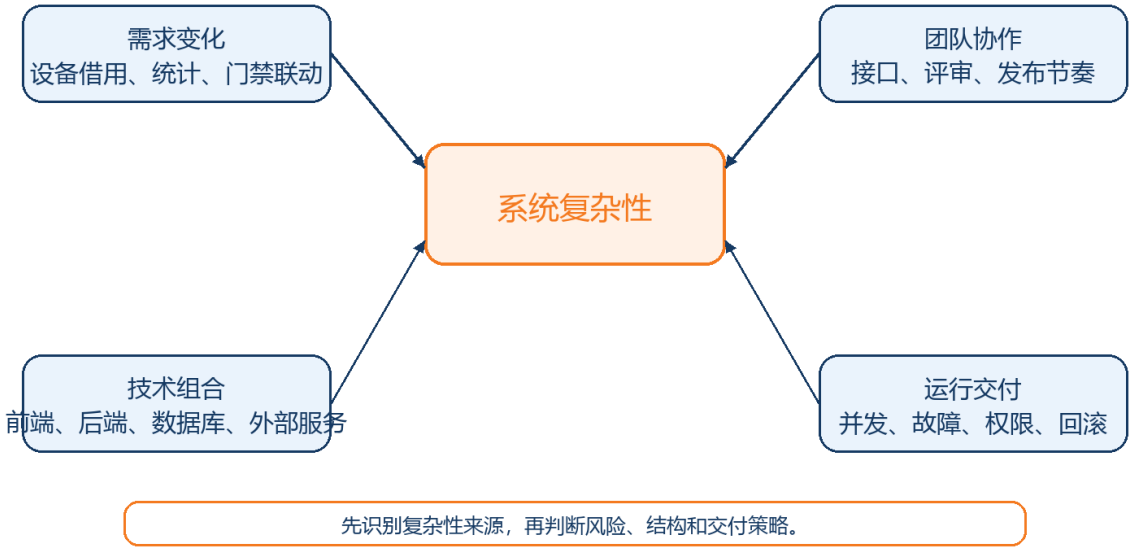
第二类是团队协作规模。个人项目可以凭记忆维护代码；三到五人的项目必须划分职责；十人以上的项目需要统一接口、分支策略、评审机制和发布流程。协作规模越大，靠“大家沟通一下”解决问题的成本越高。

第三类是技术组合深度。一个系统可能同时涉及前端、后端、数据库、缓存、消息队列、身份认证、文件存储和第三方服务。技术组件越多，故障边界越难判断，版本兼容和数据一致性问题也越容易出现。

第四类是运行与交付约束。系统不是只在演示环境中运行，还要面对并发访问、网络抖动、数据备份、权限审计、日志追踪、版本升级和回滚。运行约束会把“代码问题”放大成“工程问题”。

【易错点】复杂性不是代码行数越多才出现。一个只有几千行代码的小系统，只要涉及真实用户、权限、并发、外部服务和长期维护，也会产生工程复杂性。

图 1-1 软件复杂性的四个来源



图注：本图把复杂性拆成四个入口，便于分析一个系统为什么会从“能跑”变成“难控”。

1.4 FR、QA、C：把问题分成三类

控制复杂系统的第一步，是把需求与约束分清楚。本课程使用一个简单但有效的三分法：功能需求、质量属性和约束条件。

功能需求（Functional Requirement, FR）回答“系统要做什么”。例如，学生提交预约、教师审批、实验员配置设备、系统生成预约记录。

质量属性（Quality Attribute, QA）回答“系统要做到什么程度”。例如，预约冲突不能发生，权限判断必须可靠，常用页面响应要足够快，通知失败不能拖垮主流程。

约束条件（Constraint, C）回答“系统必须在什么边界内完成”。例如，项目周期只有八周，团队只有四人，必须使用学校统一认证，部署环境只能使用学院服务器，期末答辩前必须交付可演示版本。

三者混在一起会导致方案失真。若把“不能重复预约”当成普通功能，就可能只在前端做一次判断；若把它识别为一致性质量要求，就会在数据库约束、事务边界和并发测试中体现出来。若忽略“统一认证”这个约束，后期再接入 CAS 或统一身份系统，改动会影响权限模型和接口设计。

图 1-2 FR/QA/C 三分法

图注：本图说明 FR、QA、C 三类问题必须同时进入工程判断，不能只列功能清单。

1.5 架构时间账：今天省下的时间以后会不会还回去

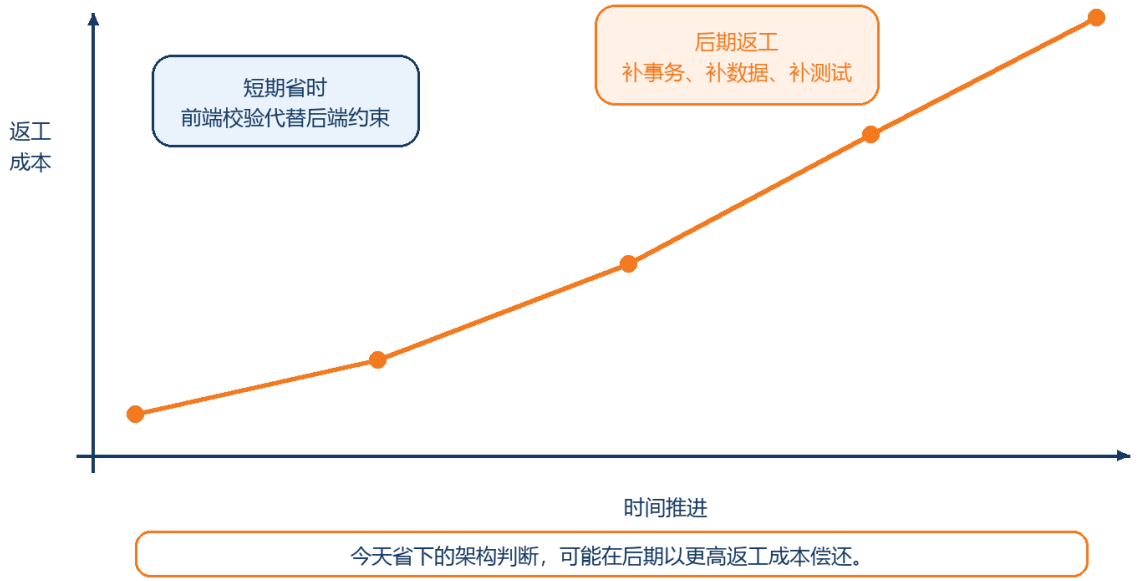
软件开发中经常出现一种错觉：为了赶进度先写一个快版本，后面再优化。这个判断并非一定错误，但必须记录代价。否则短期节省的时间，会以调试、返工、上线故障和维护困难的形式还回来。

例如，预约冲突只靠前端判断，初期可以节省后端事务设计时间；但一旦出现并发冲突，就需要补数据库约束、修改接口、清理错误数据、重新测试。又如，权限只在页面隐藏按钮，初期看起来简单；但后续发现越权访问后，就要补统一鉴权、改所有接口、增加安全测试，代价远高于一开始设计清楚。

架构时间账不是要求所有事情一开始都做到完美，而是要求清楚记录：哪些地方是有意简化，为什么暂时接受，何时需要偿还，观察什么信号。软件工程中成熟的判断，不是“绝不欠债”，而是“不让无意识的债务滚成系统风险”。

【项目提示】课程项目可以接受简化设计，但必须写清楚简化理由。例如“通知先使用同步邮件发送”可以接受，但需要说明如果邮件失败如何处理、何时改成异步、如何避免主流程被拖慢。

图 1-3 架构时间账



图注：本图用于说明架构时间账：早期省下的设计判断，可能在后期以更高返工成本偿还。

1.6 软件工程的完整视角

一个软件系统从想法到交付，通常经历需求、设计、实现、测试、发布、运行和演进。每个阶段都可能产生工程判断。

需求阶段需要识别真实问题，而不是只收集“用户想要什么功能”。设计阶段需要确定结构、边界和关键质量属性。实现阶段需要把设计转化为代码和接口。测试阶段需要验证功能、质量和安全。发布阶段需要把系统可靠地交付到运行环境。运行阶段需要监控、日志、告警、故障处理和版本回滚。演进阶段需要在新需求和旧结构之间保持平衡。

本课程后续章节会沿着这个链条展开：过程模型决定开发节奏，项目管理决定资源与风险控制，需求工程决定问题是否定义清楚，建模与架构设计决定系统如何组织，测试和交付决定系统是否可验证、可上线、可维护。

【考试提示】解释“软件工程为什么不等于写代码”时，可以从需求、设计、协作、质量、交付和演进六个方面展开，并结合一个具体系统说明。

1.7 项目实践：给你的系统做一次复杂性体检

课程项目开始前，建议先完成一页“复杂性体检卡”。

检查项	可填写问题	示例
需求变化	后续最可能增加什么功能	设备统计、门禁联动
团队协作	哪些模块需要多人同时改	预约、审批、通知
技术组合	是否依赖外部服务	统一认证、邮件服务
运行交付	哪些故障会影响用户	通知失败、重复预约

质量属性	哪些指标必须保证	不重复预约、权限可靠
约束条件	时间、人力、部署边界	四人团队、八周交付

这个体检卡不要求一次性给出所有答案。它的作用是提醒项目组：写代码前要先判断复杂性来源，并把关键问题转化为后续需求、设计、测试和交付材料。

实验室预约系统复杂性体检卡示例

检查项	示例填写	后续影响
需求变化	后续可能增加门禁联动、设备统计、导师审批	第 4 章需求规格要保留扩展边界
团队协作	前端、后端、数据库、测试至少四类工作并行	第 3 章需要任务拆解和接口约定
技术组合	依赖统一认证、邮件服务、数据库事务	第 6 章需要明确外部依赖和一致性策略
运行交付	通知失败、重复预约、权限越权会影响真实使用	第 7 章、第 8 章需要测试和运行监控
质量属性	不重复预约、权限可靠、核心流程可追踪	后续架构、测试、交付都要围绕这些质量属性展开
约束条件	课程周期有限，团队经验有限，部署环境有限	第 2 章过程模型和第 3 章计划要控制首版范围

【考试提示】导论题常考“为什么软件工程不等于写代码”。答题时可以按“复杂性来源 → FR/QA/C 分类 → 架构时间账 → 后续工程活动”组织，不要只写“软件工程包括很多阶段”。

本章与后续学习的关系

本章建立全课的复杂性视角。后续章节会分别从开发节奏、项目计划、需求、建模、架构、测试和交付七个方向处理这些复杂性。

1.8 本章小结

1. 会写代码不等于能控制复杂系统，真实系统要面对需求变化、团队协作、技术组合和运行交付约束。
2. FR、QA、C 是分析软件问题的基础分类，能够帮助项目组区分“做什么”“做到什么程度”和“在什么条件下做”。
3. 架构时间账提醒项目组记录有意简化和未来偿还条件，避免无意识技术债。
4. 软件工程是从需求到运行演进的完整活动，代码只是其中一个环节。
5. 课程项目应从复杂性体检开始，而不是直接进入页面和接口开发。

1.9 复习题与参考答案要点

【题型：名词解释｜难度：基础】基础概念题 1：什么是软件复杂性的四个来源？

参考答案要点：需求规模与变化速度、团队协作规模、技术组合深度、运行与交付约束。答题时应说明每一类复杂性如何影响项目，而不是只列名词。

【题型：名词解释｜难度：基础】基础概念题 2：FR、QA、C 分别表示什么？

参考答案要点：FR 是功能需求，回答系统要做什么；QA 是质量属性，回答做到什么程度；C 是约束条件，回答必须在什么边界内完成。

【题型：名词解释｜难度：基础】基础概念题 3：什么是架构时间账？

参考答案要点：架构时间账是对当前设计选择未来代价的记录和判断，包括简化原因、风险、偿还条件和观察指标。

【题型：简答题｜难度：中等】简答题 1：为什么“功能能跑”不代表系统工程上已经完成？

参考答案要点：功能能跑只说明局部流程可执行；工程完成还需要考虑并发、权限、可靠性、可维护性、可测试性、部署运行和后续演进。

【题型：简答题｜难度：中等】简答题 2：用实验室预约系统说明 FR、QA、C 的区别。

参考答案要点：提交预约是 FR；不能重复预约、权限可靠、响应时间可接受是 QA；必须接入统一认证、八周内交付、部署在学院服务器是 C。

【题型：案例分析｜难度：拔高】案例分析题：某学生项目把所有权限控制都放在前端页面隐藏按钮中。请分析风险并提出改进方案。

参考答案要点：风险包括越权访问、接口被直接调用、审计困难；改进方案包括后端统一鉴权、角色权限模型、接口级安全测试、日志记录和权限异常告警。

【题型：项目应用｜难度：中等】项目实践题：为你的课程项目填写一张复杂性体检卡。

参考答案要点：至少覆盖需求变化、团队协作、技术组合、运行交付、质量属性和约束条件，并说明最优先处理的两个风险。

第 2 章 软件过程模型与开发节奏选择

2.1 本章导读：开发节奏本质上是风险暴露节奏

软件过程模型常被理解为“瀑布、迭代、敏捷”等名词集合。更有价值的理解方式是：过程模型决定团队在什么时候暴露问题、什么时候获得反馈、什么时候冻结范围、什么时候调整方向。

在实验室预约与设备管理系统中，若需求稳定、接口明确、交付周期较长，可以采用更重的前期分析与阶段评审；若用户对预约流程、审批规则和通知方式还没有形成清晰认识，就需要尽早做原型并持续获得反馈；若系统涉及并发冲突、权限安全、外部认证等高风险点，就不能把关键风险留到最后验收才发现。

因此，本章不把过程模型作为背诵对象，而是训练一种判断能力：面对不同复杂性来源，应选择怎样的开发节奏；如果选错节奏，会付出什么代价。

【你要记住】过程模型不是流程图好不好看，而是风险、反馈和交付节奏是否匹配项目条件。

2.2 一个反面案例：把不确定项目当成确定项目做

某项目组计划开发“游戏领域社交网站”。设想很吸引人：玩家可以发布攻略、组队、评价游戏、关注好友，还能根据兴趣推荐内容。项目组一开始按照严格阶段推进：第一周写完需求文档，第二周设计数据库，第三周开始编码，第八周集成测试。

问题在第四周开始暴露。用户并不确定最想要的是攻略社区、组队工具还是游戏评价平台；推荐功能需要数据积累，短期无法验证；组队功能涉及实时状态和通知，技术复杂度被低估；页面做出来后，试用者发现“更快找到队友”才是真需求，而不是“更多帖子分类”。

这个项目的问题不是成员不努力，而是把高不确定性项目当成低不确定性项目管理。需求还没有被验证，就提前冻结范围；关键风险没有早做实验，却把它留到编码后期；用户反馈来得太晚，导致大量工作变成返工。

对实验室预约系统也一样。若一开始不了解教师审批习惯、实验员设备管理规则、学院统计口径，就直接写完整后台，很可能做出“能用但不合用”的系统。

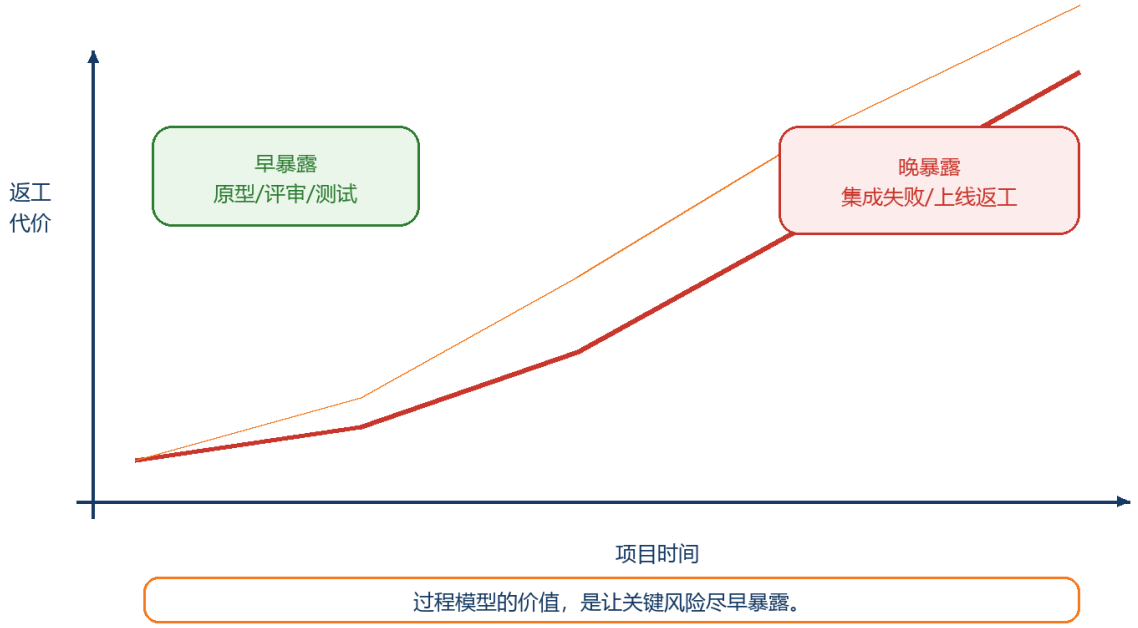
2.3 瀑布模型：适合需求稳定但害怕晚发现问题

瀑布模型把开发活动分成需求、设计、实现、测试、交付等阶段，强调阶段产物和评审。它的优点是管理边界清楚，适合需求相对稳定、合规文档要求较高、变更成本可控的项目。

但瀑布模型最典型的风险是反馈来得晚。若需求理解错了，往往要到测试或验收阶段才发现；若设计方案不合适，返工成本也已经升高。瀑布不是错误模型，它的问题在于被用于高不确定性项目时，容易把问题推迟暴露。

在实验室预约系统中，若学院已经有明确纸质流程、审批规则稳定、数据字段和报表口径清楚，那么阶段化开发可以降低管理混乱。但如果审批规则还在讨论、设备状态含义不统一、用户界面需要试用反馈，就不适合完全冻结需求后再开发。

图 2-1 问题暴露时间与返工代价



图注：本图说明不同开发节奏会影响问题暴露时机，暴露越晚，沟通、返工和交付风险越高。

2.4 原型模型：先让用户看见，再判断是不是想要的

原型模型适用于用户说不清需求、界面交互重要、业务流程需要反复确认的场景。原型不一定是完整系统，可以是纸面流程、低保真界面、可点击页面，也可以是一个只覆盖关键流程的小程序。

实验室预约系统最适合先做的原型，不是完整后台，而是“学生提交预约—教师审批—学生查看结果”的最短闭环。原型要验证的问题包括：学生如何选择设备和时间？审批人需要看到哪些信息？冲突提示放在哪里？预约状态怎样表达才不误解？

原型模型的风险在于，原型容易被误认为正式系统。若原型代码质量很低，却被直接扩展成产品，后续会积累大量技术债。因此原型必须明确目的：它是为了验证需求和交互，不是为了绕过设计。

【易错点】原型不是“先随便写一个以后再说”。原型必须说明验证问题、保留范围和废弃条件。

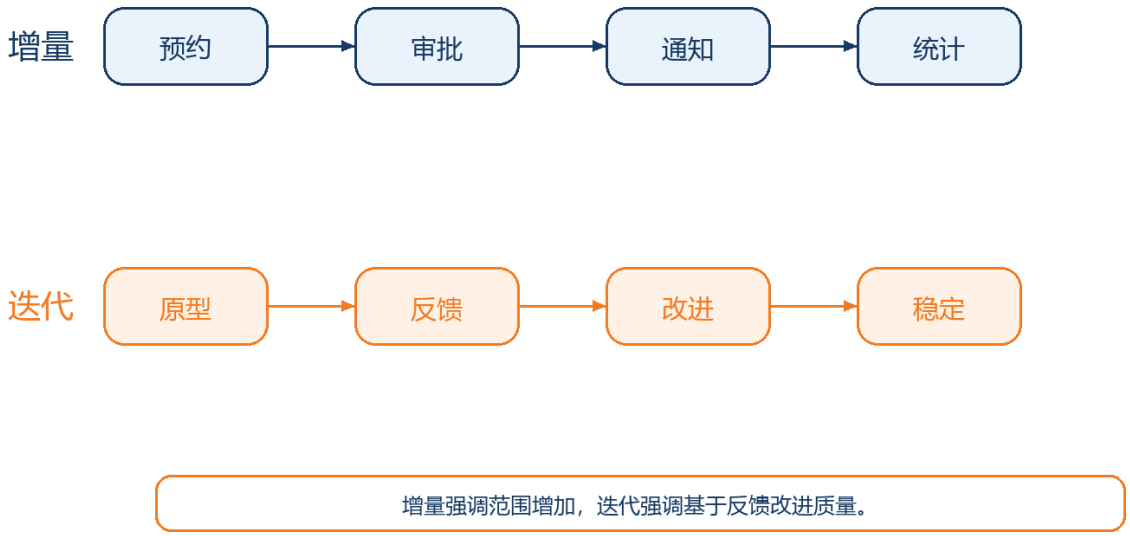
2.5 增量与迭代：一次交付一部分，反复修正同一部分

增量模型强调把系统分成若干可交付部分，每次增加新的功能范围。迭代模型强调对已有部分反复改进，每轮根据反馈修正需求、设计和实现。实际项目中，二者常常结合使用：每个版本交付一个增量，同时在增量内部进行迭代。

实验室预约系统可以按增量组织：第一增量完成预约提交和审批；第二增量加入设备管理；第三增量加入通知与统计；第四增量接入统一认证。每个增量内部可以迭代改进，例如预约页面第一版只支持按设备预约，第二版支持按时间段筛选，第三版增加冲突提示。

增量与迭代的价值在于让系统早进入可运行状态，同时保留调整空间。风险在于，如果没有版本边界和优先级，项目会变成“每周都改一点，但没有稳定成果”。

图 2-2 增量与迭代的区别



图注：本图帮助区分增量和迭代：前者强调范围扩展，后者强调反馈驱动的质量改进。

2.6 螺旋模型、V 模型与 RUP：风险、验证和架构中心

螺旋模型把每轮开发看成“目标设定—风险分析—工程实现—评审计划”的循环。它适合风险高、技术不确定性强、需要持续评估成本与收益的项目。例如，若实验室预约系统要接入门禁硬件、设备传感器和统一身份认证，就应先验证接口、权限和故障处理，而不是直接开发完整业务。

V 模型强调开发活动与测试活动的对应关系。需求对应验收测试，系统设计对应系统测试，详细设计对应集成测试，编码对应单元测试。它适合对验证要求较高的项目，能提醒团队在设计阶段就考虑后续如何测试。

RUP（Rational Unified Process，统一软件开发过程）强调迭代式、用例驱动和架构中心。它不是简单的“重文档流程”，而是要求项目在早期识别关键用例和架构风险，并通过多轮迭代逐步稳定系统结构。对课程项目而言，不需要完整采用 RUP，但可以吸收两个思想：用核心用例驱动需求和测试，用架构基线控制后续扩展。

这三类思想可以帮助项目组避免两个常见问题：只顾写功能不识别风险，以及到最后才考虑测试。对课程项目而言，不需要完整套用复杂流程，但应吸收“风险先验证、设计对应测试、核心架构早稳定”的思想。

2.7 敏捷与 Scrum：用短周期把反馈变成节奏

敏捷开发强调快速反馈、持续交付、拥抱变化和团队协作。Scrum 是常见敏捷框架之一，通常包括产品待办列表（Product Backlog）、冲刺待办列表（Sprint Backlog）、短周期冲刺（Sprint）、每日站会、评审和回顾。

Product Backlog 是所有待做事项的有序列表；Sprint Backlog 是本轮冲刺承诺完成的事项；时间盒是固定周期限制，例如两周一轮；燃尽图用于观察剩余工作变化；看板用于展示任务流动；估算扑克帮助团队对工作量形成相对估计。

在实验室预约系统中，一个 Sprint 可以聚焦“预约闭环”。任务可以拆成：预约页面、冲突校验接口、审批页面、状态流转、基础测试和演示数据。每日站会不应变成流水账，而应回答：昨天完成了什么，今天要推进什么，遇到什么阻塞。

【项目提示】课程项目使用敏捷思想时，不必形式复杂，但必须有可见任务板、稳定迭代节奏、可演示增量和每轮复盘。

2.8 模型选择：五个典型场景

过程模型选择可以从五类场景判断。重点不是记住模型名称，而是判断主要不确定性在哪里、错误节奏会造成什么代价。

需求稳定、合规文档强
主要特点：变更少，阶段产物重要
错误选择的代价：若过度敏捷，可能缺少评审和文档依据
推荐节奏：瀑布或阶段化迭代

用户说不清需求
主要特点：交互和流程需确认
错误选择的代价：过早冻结需求，后期大量返工
推荐节奏：原型 + 迭代

技术风险高
主要特点：外部系统、硬件、性能风险突出
错误选择的代价：技术风险留到后期，可能整体推翻
推荐节奏：螺旋思想，先验证风险

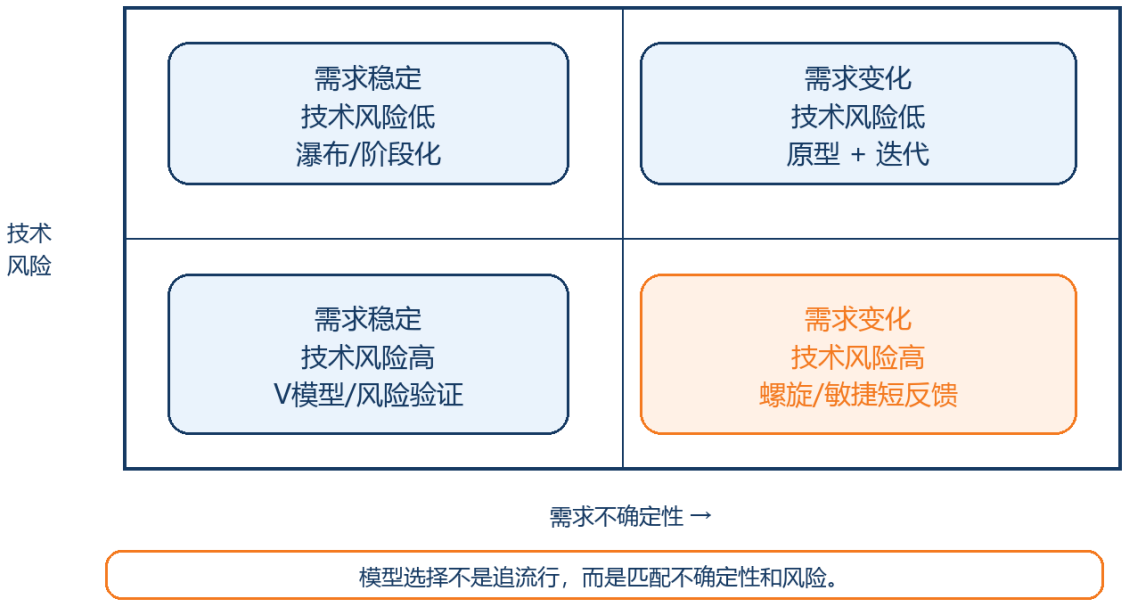
验证要求高
主要特点：质量、安全、验收标准明确
错误选择的代价：测试后置，发现问题太晚
推荐节奏：V 模型思想

团队小、周期短、变化快
主要特点：需要持续反馈和演示
错误选择的代价：计划过重，成果不可见，反馈过晚
推荐节奏：敏捷/Scrum 简化实践

例如，实验室预约系统若首版只面向一个学院内部使用，团队人数少、周期短，就可以采用“原型澄清需求 + 增量交付 + 关键风险提前验证”的混合节奏。若后续变成全校平台，涉及统一认证、门禁、跨学院规则和数据合规，就要增加架构基线、接口评审、测试门禁和发布管理。

【项目提示】课程项目选择过程模型时，建议写成三句话：项目最不确定的是什么；本轮怎样提前验证；每个版本交付什么可演示成果。

图 2-3 开发节奏选择矩阵



图注：本图把需求不确定性和技术风险放在同一张矩阵中，用于判断更合适的开发节奏。

2.9 项目实践：为课程项目设计开发节奏

课程项目建议用一页“开发节奏说明”描述过程选择。

项目条件	填写提示
需求确定性	哪些需求明确，哪些需要原型验证
技术风险	哪些接口、性能、权限或并发问题最不确定
版本节奏	每两周或每三周交付什么可演示增量
反馈机制	谁来试用，如何记录反馈
测试对应	每个关键设计如何验证
调整规则	哪些情况触发范围调整

一个合理答案可以是：首轮用原型验证预约与审批流程；第二轮交付预约闭环；第三轮补设备管理和通知；第四轮做权限、测试和部署；并把重复预约、统一认证、通知失败列为早期风险验证项。

本章与后续学习的关系

本章回答如何选择开发节奏。下一章将进一步讨论如何把节奏选择落到项目计划、风险登记和协作规范中。

2.10 本章小结

1. 过程模型决定风险暴露和反馈获得的节奏。
2. 瀑布适合需求稳定和阶段产物明确的场景，但不适合高不确定性项目。
3. 原型、增量、迭代和敏捷可以帮助项目组尽早验证需求和交付价值。
4. 螺旋模型提醒团队先处理高风险点，V 模型提醒设计与测试要对应。
5. 课程项目应选择轻量、可执行、有反馈的开发节奏，而不是机械套用某个模型。

2.11 复习题与参考答案要点

【题型：名词解释｜难度：基础】基础概念题 1：瀑布模型的主要特点是什么？

参考答案要点：阶段清晰、文档和评审明确、适合需求稳定项目；主要风险是反馈晚、变更成本高。

【题型：名词解释｜难度：基础】基础概念题 2：增量模型和迭代模型有什么区别？

参考答案要点：增量强调每次增加可交付功能范围；迭代强调对已有功能反复改进。实际项目常结合使用。

【题型：名词解释｜难度：基础】基础概念题 3：Scrum 中 Product Backlog 和 Sprint Backlog 的区别是什么？

参考答案要点：Product Backlog 是项目所有待办事项的有序列表；Sprint Backlog 是本轮冲刺承诺完成的事项集合。

【题型：简答题｜难度：中等】简答题 1：为什么高不确定性项目不宜完全采用瀑布式节奏？

参考答案要点：需求、交互、技术风险无法在前期一次性确定；若反馈晚，会造成大量返工；应通过原型、迭代或风险验证提前暴露问题。

【题型：简答题｜难度：中等】简答题 2：用实验室预约系统说明如何混合使用多种过程思想。

参考答案要点：用原型验证预约流程，用增量组织版本，用 V 模型思想安排测试，用敏捷节奏做反馈，用螺旋思想提前验证并发和统一认证风险。

【题型：案例分析｜难度：拔高】案例分析题：某团队第七周才发现用户不需要复杂社区分类，而需要快速组队入口。请分析过程模型问题并给出改进。

参考答案要点：问题是需求验证太晚、反馈机制缺失、过早冻结范围；改进包括低保真原型、早期用户试用、短周期迭代、优先验证核心价值。

【题型：项目应用｜难度：中等】项目实践题：为你的课程项目制定一个四轮开发节奏。

参考答案要点：每轮应有目标、可演示成果、主要风险、反馈对象和调整规则。

第 3 章 项目管理、可行性分析与风险控制

3.1 本章导读：项目管理不是填表，而是让目标可执行

软件项目失败时，问题往往不只出在技术。目标没有对齐、范围不断膨胀、任务无人负责、进度不可见、风险没有提前处理，都会让一个看似简单的系统变得难以收尾。

实验室预约与设备管理系统可以被写成一句话：“做一个预约系统”。但这句话无法指导项目执行。项目组需要进一步回答：服务哪些用户？首版交付哪些功能？哪些功能不做？哪些风险先验证？任务如何拆分？什么时候集成？如果统一认证接入失败怎么办？如果团队成员进度不一致怎么办？

项目管理的价值，是把模糊目标转化为可执行、可跟踪、可调整的工程活动。本章关注可行性分析、项目计划、任务拆解、风险控制、沟通机制和工具使用。

【你要记住】项目管理不是额外负担，而是让团队知道“现在做什么、谁负责、何时交付、风险在哪里”的工程机制。

3.2 一个反面案例：没有计划的“最后一周集成”

某项目组四名成员分工明确：一人做前端，一人做后端，一人做数据库，一人做文档。前几周每个人都在“推进自己的部分”，但没有统一接口约定，也没有阶段集成。到最后一周，前端字段名和后端接口不一致，数据库状态枚举和页面显示不匹配，通知功能没有测试环境，项目文档也无法反映真实实现。

更麻烦的是，团队没有风险清单。统一认证接口能否接入、预约冲突如何处理、文件上传是否需要权限限制、统计报表是否要导出，这些问题都在临近答辩时集中出现。项目不是没有人做事，而是缺少把工作组织起来的机制。

改进方式不是增加复杂管理工具，而是尽早建立四类基本产物：范围边界、里程碑计划、任务看板、风险清单。它们让团队能及时发现“哪些工作没有主人”“哪些假设还没有验证”“哪些接口必须提前对齐”。

3.3 可行性分析：先判断做不做得成

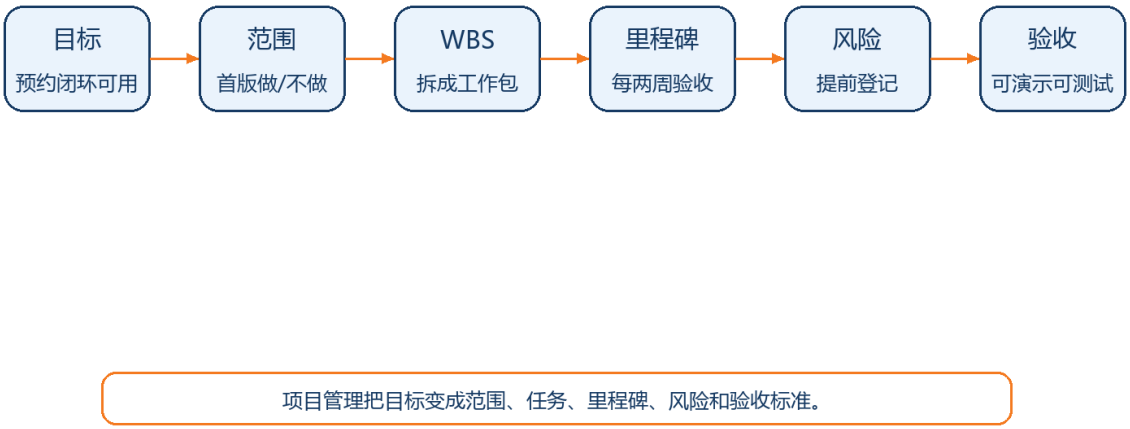
可行性分析不是大型项目才需要。课程项目也应在开工前判断技术、时间、资源和业务可行性。

技术可行性关注团队是否具备实现关键功能的能力。例如，是否会处理并发预约冲突，是否能接入统一认证，是否能部署后端服务。时间可行性关注任务规模是否匹配学期周期。资源可行性关注人力、测试数据、服务器和外部接口是否可获得。业务可行性关注系统是否真正解决目标用户问题。

对实验室预约系统而言，首版可行方案应避免把所有设想一次做完。可以先保证预约、审批、状态流转、基础权限和数据记录；把门禁联动、复杂统计、设备传感器接入放入后续版本或扩展目标。

【易错点】可行性分析不是证明“项目一定能做”，而是识别哪些目标必须缩小、哪些风险必须提前验证。

图 3-1 从目标到可执行计划



图注：本图说明项目管理的主线：目标必须落到范围、任务、里程碑、风险和验收标准上。

3.4 范围管理：先说清楚不做什么

范围管理的关键不只是列出要做功能，还要明确首版不做什么。对课程项目来说，范围越模糊，越容易出现“每个人都觉得应该加一点”的范围蔓延。

例如，实验室预约系统首版可以定义为：学生预约、教师审批、实验员管理设备、系统发送基础通知、管理员查看预约记录。暂不做移动端 App、门禁硬件联动、复杂推荐算法、跨学院共享、自动排课。这个边界不是降低要求，而是保证核心业务闭环可交付。

范围边界要与验收标准绑定。若写“设备管理”，应说明包括设备新增、停用、维护状态和可预约状态；若写“通知”，应说明通知触发条件、失败处理和是否影响主流程。

【项目提示】项目计划中建议加入“本版本不做清单”。它能减少后期争论，也能体现团队对可交付范围的控制能力。

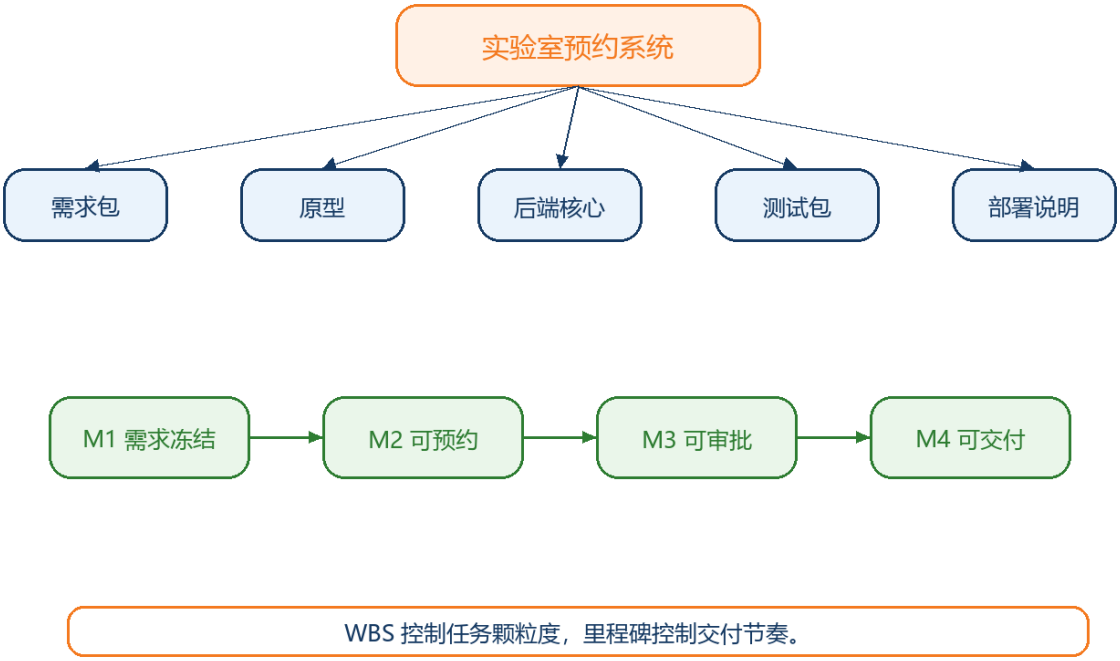
3.5 WBS 与里程碑：把大目标拆成可交付单元

WBS（工作分解结构）用于把项目目标拆成较小的工作包。好的工作包应具备清晰产出、明确负责人、可估算工作量、可验收标准。

实验室预约系统可以拆成：用户与权限、设备管理、预约申请、审批流、冲突校验、通知服务、统计报表、部署与测试、项目文档。每个工作包再拆为接口、页面、数据表、测试用例和文档。

里程碑不是“第几周写代码”这种模糊描述，而是可验证状态。例如：第 2 周完成预约流程原型评审；第 4 周完成预约闭环可演示；第 6 周完成权限与通知集成；第 8 周完成测试、部署和答辩材料。

图 3-2 WBS 与里程碑路径



图注：本图展示工作分解与交付时间线如何结合，避免所有工作堆到最后一周集成。

实验室预约系统 WBS 片段

预约申请
子任务：预约表单、冲突校验、提交接口
负责人：后端 A/前端 B
产出物：页面、接口、测试用例
验收方式：能提交待审批预约，冲突时给出提示

审批流程
子任务：审批列表、审批详情、状态流转
负责人：后端 C/前端 B
产出物：审批页面、状态机说明
验收方式：审批通过/拒绝后状态正确

设备管理
子任务：设备新增、停用、维护状态
负责人：后端 A
产出物：数据表、管理页面
验收方式：维护中设备不可预约

通知服务
子任务：通知任务、失败记录、重试说明
负责人：后端 C
产出物：通知模块、日志
验收方式：邮件失败不影响预约保存

测试与部署
子任务：测试数据、部署说明、演示环境
负责人：全组
产出物：测试报告、部署文档
验收方式：从零部署可运行

八周里程碑样例

周次	里程碑	可验收成果
第 2 周	原型与需求评审	预约主流程原型、核心用例、范围边界
第 4 周	预约闭环	学生提交、审批人审批、状态可查询
第 6 周	权限、通知与测试	角色权限、通知失败处理、核心测试用例
第 8 周	部署与答辩	可运行系统、部署说明、项目文档、演示脚本

【项目提示】WBS 不要按“前端、后端、数据库”粗分到底，而要落实到可验收的业务成果。否则每个人都在做事，但系统闭环迟迟无法出现。

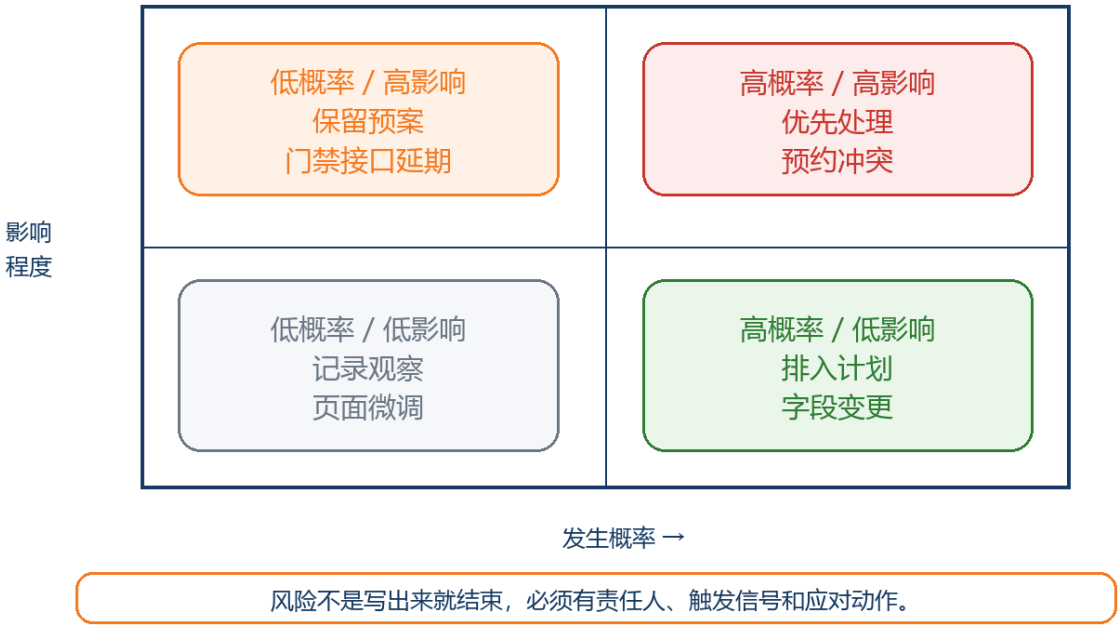
3.6 风险管理：把“可能出事”写成可跟踪事项

风险管理不等于写几句“时间不够、技术难度大”。有效风险项至少包括：风险描述、触发条件、影响范围、概率、影响程度、应对措施、负责人和观察信号。

实验室预约系统常见风险包括：并发预约导致冲突；统一认证接口接入失败；通知服务不可用；审批规则临时变化；团队成员接口约定不一致；测试数据不足。每个风险都应对应具体动作。例如“统一认证接入失败”的应对措施可以是先定义认证适配接口，准备本地账号模拟方案，并在第 3 周前完成接口验证。

风险矩阵可以把概率和影响结合起来。高概率高影响风险必须提前处理；低概率低影响风险可以记录观察；高影响但概率不确定的风险应做技术验证。

图 3-3 风险矩阵



图注：本图用概率和影响两个维度对项目风险排序，帮助项目组安排负责人和处理优先级。

风险登记表示例

统一认证接入失败
触发信号：第 3 周仍无法获得测试接口
影响：登录与权限受影响
概率/影响：中/高
应对措施：先设计认证适配接口，准备本地账号模拟
负责人：后端 A
关闭时间：第 3 周

重复预约
触发信号：并发测试出现同一时段两条有效记录
影响：核心业务失效
概率/影响：中/高
应对措施：增加数据库唯一约束和事务测试
负责人：后端 C
关闭时间：第 5 周

通知服务不稳定
触发信号：邮件发送超时或失败
影响：用户无法及时获知状态
概率/影响：高/中
应对措施：主流程与通知解耦，记录失败任务
负责人：后端 C
关闭时间：第 6 周

前后端接口不一致
触发信号：联调时字段频繁改名
影响：集成延期
概率/影响：高/中
应对措施：每周接口评审，变更写入 Ticket
负责人：全组
关闭时间：每周

测试数据不足
触发信号：演示时缺少多角色数据
影响：验收效果差
概率/影响：中/中
应对措施：准备初始化脚本和测试账号
负责人：测试/文档
关闭时间：第 7 周

3.7 沟通与会议：让信息流动，而不是制造形式

软件项目中的会议只有在形成决策、暴露阻塞、同步状态或沉淀复盘时才有价值。常见会议包括立项会、需求评审、方案评审、每日站会、版本评审和复盘会。

立项会要对齐目标、范围和角色；需求评审要确认用户、场景和验收标准；方案评审要暴露架构风险和放弃方案；站会要解决阻塞而不是汇报流水账；复盘会要记录做得好的地方、出现的问题和下一轮改进。

对课程项目而言，推荐使用轻量机制：每周一次版本会，记录本周完成、下周计划、风险变化和待决策问题；关键设计变更必须形成记录，不要只停留在聊天记录里。

3.8 工具链：工具服务于透明度

工具不是项目管理的核心，但合适工具能提高透明度。任务看板用于展示待做、进行中、待评审、已完成；Issue 或 Ticket 用于记录任务、缺陷和需求变更；Pull Request 用于代码评审和合并；CI 用于自动构建和测试。

一个小团队可以使用 Git 仓库、看板、文档协作工具和简单的版本记录。重要的不是工具名称，而是信息是否可追踪：任务是否有负责人，接口变更是否有记录，缺陷是否能复现，版本是否能回退。

【考试提示】解释项目管理工具作用时，不要只列工具名，应说明它们如何支持任务可见、协作可追踪、质量可验证和发布可控制。

轻量 Ticket 与 PR 样例

类型	示例
Ticket 标题	增加预约提交接口的冲突校验
Ticket 内容	背景：当前接口只保存预约；要求：同一设备同一时段只能存在一条有效预约；验收：并发提交时只能成功一条，其余返回冲突提示
PR 检查项	是否更新接口说明；是否补充冲突测试；是否影响审批状态；是否有数据库迁移脚本
CI 检查项	后端测试通过；前端构建通过；数据库初始化脚本可执行

【考试提示】项目管理题如果问“工具如何支持协作”，答题应落到任务可追踪、接口可评审、代码可合并、质量可验证，而不是只列工具名称。

3.9 项目计划样例：实验室预约系统

一个简化项目计划可以包含以下元素。

元素	示例
项目目标	完成预约、审批、设备管理、通知和基础统计
首版范围	Web 端，支持三类角色，完成预约闭环
不做范围	门禁硬件联动、移动 App、复杂推荐
里程碑	原型评审、预约闭环、权限通知集成、部署验收
风险	并发冲突、统一认证、通知失败、团队接口不一致
质量要求	不重复预约、权限可靠、核心流程可测试
交付物	PPT、需求说明、架构图、测试报告、部署说明

这类计划不追求复杂，但必须能让项目组回答：现在处在哪个阶段，下一步交付什么，哪些风险还没有关闭。

本章与后续学习的关系

本章把目标、范围、计划和风险转化为可执行管理动作。下一章将进一步进入需求工程，明确系统到底要解决什么问题。

3.10 本章小结

1. 项目管理的核心是让目标、范围、任务、风险和交付状态可见。
2. 可行性分析帮助团队提前判断哪些目标应收缩，哪些风险应验证。
3. WBS 和里程碑把大目标拆成可交付单元，避免最后一周集成。
4. 风险管理要写成可跟踪事项，而不是泛泛而谈。
5. 工具链的价值在于提高透明度和可追踪性。

3.11 复习题与参考答案要点

【题型：名词解释｜难度：基础】基础概念题 1：什么是可行性分析？

参考答案要点：从技术、时间、资源、业务等方面判断项目目标是否可实现，并识别需要缩小或提前验证的风险。

【题型：名词解释｜难度：基础】基础概念题 2：WBS 的作用是什么？

参考答案要点：把项目目标拆分为可负责人、可估算、可验收的工作包，支持计划、协作和跟踪。

【题型：名词解释｜难度：基础】基础概念题 3：风险项应包含哪些信息？

参考答案要点：风险描述、触发条件、影响范围、概率、影响程度、应对措施、负责人和观察信号。

【题型：简答题｜难度：中等】简答题 1：为什么项目计划中要写“不做范围”？

参考答案要点：防止范围蔓延，明确首版交付边界，减少团队争议，保证核心闭环按时完成。

【题型：简答题｜难度：中等】简答题 2：说明任务看板、Ticket、PR、CI 分别如何支持项目管理。

参考答案要点：看板显示状态，Ticket 记录任务和缺陷，PR 支持评审和合并，CI 自动构建测试并暴露质量问题。

【题型：案例分析｜难度：拔高】案例分析题：某团队最后一周才发现前后端接口不一致。请从项目管理角度分析原因和改进。

参考答案要点：原因包括缺少接口评审、里程碑集成、任务跟踪和风险管理；改进包括接口契约、早期集成、版本评审、CI 和任务责任人。

【题型：项目应用｜难度：中等】项目实践题：为课程项目编写一页风险清单。

参考答案要点：至少列出 5 项风险，每项包含影响、触发条件、应对措施、负责人和关闭时间。

第 4 章 需求工程：从用户声音到可验证规格

4.1 本章导读：需求不是用户说的一句话

需求工程的难点在于，用户表达的往往是愿望、抱怨或解决方案，而不是经过分析的真实需求。软件工程师需要把这些声音转化为目标用户、使用场景、真实问题、系统边界和可验证规格。

实验室预约系统中，学生可能说“希望预约更快”，教师可能说“希望安排更清楚”，实验员可能说“希望少沟通”，学院管理者可能说“希望设备利用率可统计”。这些话都重要，但它们不能直接变成系统设计。需要继续追问：谁在什么情况下使用？当前痛点是什么？成功标准是什么？哪些规则必须满足？哪些信息必须记录？哪些质量属性不可妥协？

本章从利益相关者、需求获取、典型用户、用例分析、规格说明和原型设计几个方面，说明如何把模糊声音变成可开发、可测试、可评审的需求材料。

【你要记住】需求分析不是收集功能清单，而是把用户声音转化为可验证的系统约定。

4.2 一个反面案例：“更快的马车”式需求

假设学生反馈：“预约设备太麻烦，希望做一个更快的预约按钮。”如果项目组直接理解为“在首页放一个一键预约按钮”，可能很快做出功能。但试用后发现问题更多：学生不知道哪台设备可用，教师不清楚申请理由，实验员仍然要反复确认冲突，系统还可能自动提交错误预约。

“更快的预约”只是表层表达。继续追问后，真实需求可能包括：学生希望少填写重复信息；系统能提前过滤不可用设备；申请理由模板化；冲突在提交前提示；预约状态及时反馈。也可能发现不同用户的“快”含义不同：学生关注提交快，实验员关注审批清楚，教师关注安排可见，管理员关注规则可控。

需求工程的价值就在这里：不把用户的第一句话当作最终答案，而是通过场景、用户、问题和规则分析出真正需要解决的系统问题。

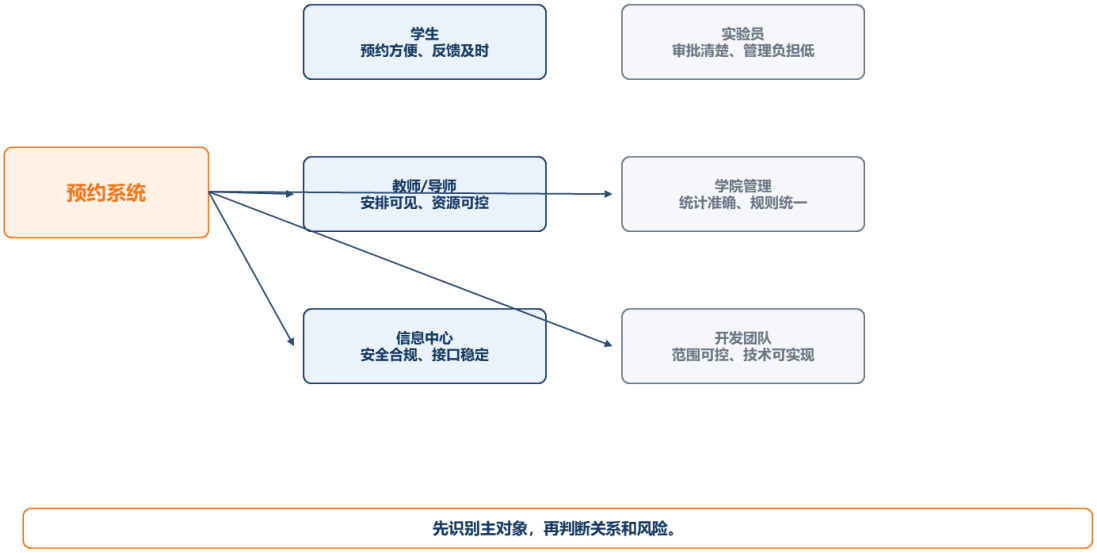
4.3 利益相关者：先识别谁会受影响

利益相关者不仅包括直接使用系统的人，也包括受系统结果影响、维护系统、监管系统或为系统提供外部能力的人。

在实验室预约系统中，学生是直接用户，教师或导师关注实验安排，实验员负责设备开放和审批，学院管理员关注统计和规则，信息中心关注统一认证和安全合规，开发团队关注实现成本和维护边界。若只听学生需求，就可能忽略审批规则、设备维护和数据安全；若只听管理者需求，又可能做出学生难以使用的系统。

利益相关者分析通常要记录：角色、目标、痛点、关心指标、可能冲突和决策权。需求不是投票结果，而是在多个利益之间寻找可实现的平衡。

图 4-1 实验室预约系统利益相关者地图



图注：本图说明需求来源于多方诉求交汇，而不是某一个用户的单方面声音。

4.4 需求获取：问问题要问到场景里

需求获取可以采用访谈、问卷、观察、文档分析、原型试用和工作坊等方式。关键不是方法名称，而是能否获得真实场景。

访谈时应避免只问“你想要什么功能”，而应问：你现在如何预约？最麻烦的一步是什么？发生冲突时怎么处理？哪些信息必须审批人看到？哪些情况必须拒绝？系统出错时你希望看到什么提示？

观察能发现用户没有说出的流程。例如，实验员可能口头说“审批很简单”，但实际会先看设备状态、实验课程安排、维护计划、申请理由和导师信息。文档分析能发现制度要求，例如开放时间、借用规则、责任归属和安全要求。原型试用能帮助用户发现自己真正关心的交互细节。

【易错点】问卷适合收集趋势，不适合单独定义复杂流程。关键业务规则应通过访谈、观察和原型反复确认。

4.5 典型用户与用户故事

典型用户（Persona）用于描述一类用户的目标、行为和痛点。它不是虚构人物传记，而是帮助团队保持需求判断的一致参照。

例如，“研一学生小李”需要快速查看设备空闲时间，提交预约并及时知道结果；“实验员王老师”需要审批清楚、减少重复沟通、避免设备冲突；“学院管理员”需要统计设备利用率并设置规则。

用户故事通常写成：“作为某类用户，我希望完成某个目标，以便获得某种价值。”例如：“作为研究生，我希望在预约前看到设备可用时间段，以便避免提交无效申请。”用户故事后面必须配验收条件，否则容易停留在口号。

【项目提示】每个核心用户故事至少写 3 条验收条件。例如“显示可用时间段”应说明数据来源、冲突规则、刷新时机和异常提示。

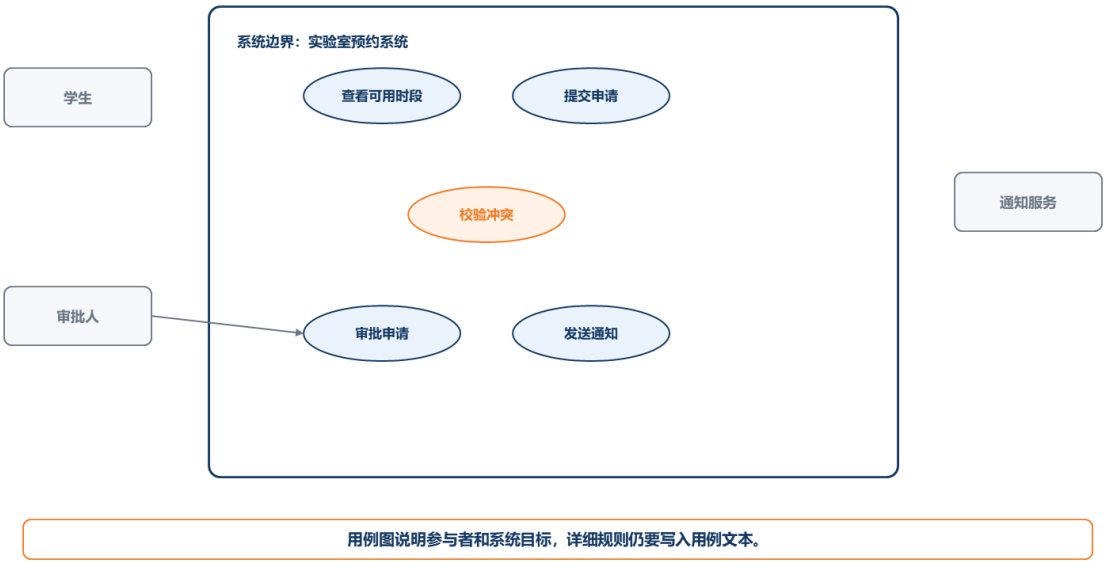
4.6 用例分析：把交互过程说完整

用例描述用户与系统之间为达成目标而发生的一组交互。一个好的用例不仅写主成功场景，还要写前置条件、后置结果、异常流程和业务规则。

以“提交实验室预约申请”为例。前置条件是用户已登录且具备预约权限；主成功场景包括选择实验室或设备、选择时间段、填写用途、系统校验冲突、提交申请、生成待审批记录、发送通知；异常流程包括时间段不可用、设备维护中、用户权限不足、网络或通知失败；后置结果是预约记录进入待审批状态，并可被审批人查看。

这种写法比“用户提交预约”更可开发，也更可测试。开发人员知道要实现哪些接口，测试人员知道要验证哪些异常，设计人员知道状态如何变化。

图 4-2 提交预约申请用例图



图注：本图强调用例图只说明系统边界、参与者和目标，详细规则仍需要用例文本补充。

提交预约申请用例文本样例

项目	内容
用例名称	提交实验室预约申请
主要参与者	学生
前置条件	学生已登录，具备预约权限；设备处于可预约状态
主成功场景	学生选择设备和时间段；填写用途和导师信息；系统校验权限、设备状态和时间冲突；系统生成待审批记录；系统通知审批人
异常流程	时间段冲突时提示重新选择；设备维护中时不可提交；权限不足时返回拒绝；通知失败时记录失败任务但不撤销预约记录
后置结果	预约进入待审批状态，并可在学生端和审批端查询

业务规则	同一设备同一时段只能有一条有效预约；取消后释放时间段；审批操作必须记录操作者和时间

【项目提示】用例文本要能支撑开发和测试。若异常流程写不出来，通常说明需求还没有分析清楚。

4.7 需求规格说明：把需求写到可验证

需求规格说明书（SRS）不是越厚越好，而是要把关键需求写清楚、写一致、写到可验证。常见内容包括：项目背景、目标用户、系统范围、功能需求、数据需求、质量属性、约束条件、接口需求、验收标准和术语表。

可验证是需求规格的核心。模糊需求如“系统响应要快”无法直接测试；可验证写法应改成“预约列表在 1000 条记录以内，常规查询响应时间不超过 2 秒”。“权限要安全”也过于笼统，可写成“学生不得访问审批接口；未授权访问返回 403；所有审批操作记录操作者、时间和结果”。

需求规格还要保持一致性。若用例中写“取消预约后释放时间段”，状态机和数据库字段也应支持取消状态；若质量属性要求“预约不能重复”，并发测试和数据库约束必须体现。

图 4-3 需求规格说明书结构

图注：本图把需求规格说明书拆成若干可检查部分，底层目标是形成可验证规格。

可验证规格示例

模糊写法	可验证写法
系统不能重复预约	对同一设备、同一时间段，数据库中最多只能存在一条状态为待审批或已通过的预约记录；并发提交时只有一个请求成功
权限要安全	学生不得访问审批接口；未授权请求返回 403；所有审批操作记录操作者、时间、结果和 IP

通知要可靠	预约状态变化后生成通知任务；通知失败不影响预约状态；失败任务记录原因并允许重试

【考试提示】“可验证需求”通常要写出条件、响应和度量。只写“快、稳定、安全、好用”不能作为合格答案。

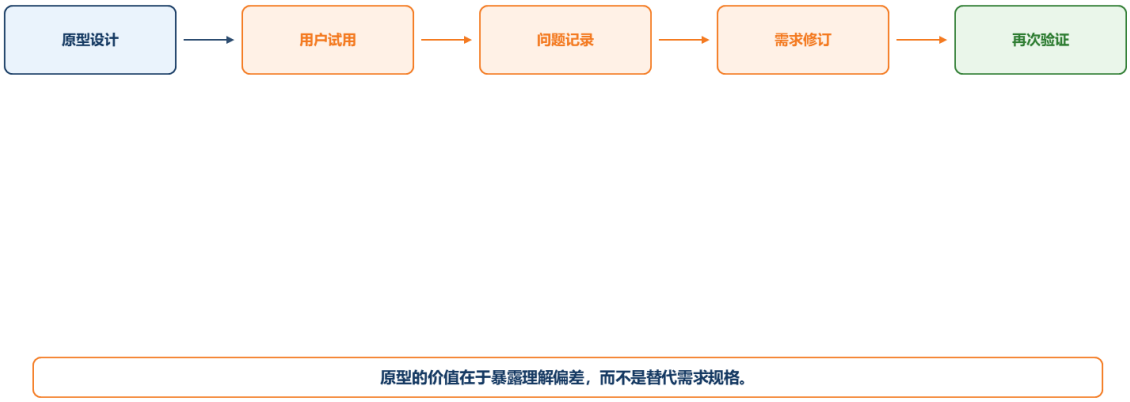
4.8 原型设计：用低成本暴露理解偏差

原型设计适用于流程、界面和交互不确定的场景。低保真原型可以快速讨论布局和流程，高保真原型可以验证交互细节，但不应过早追求视觉完整。

实验室预约系统的原型应重点展示四个页面：设备列表、可用时间选择、预约申请表、审批详情。原型评审时要观察用户是否能找到可用设备，是否理解状态含义，是否知道失败原因，审批人是否能判断申请是否合理。

原型反馈要转化为需求修改，而不是停留在“用户觉得还可以”。例如，若用户多次误解“待确认”和“待审批”，就应调整状态命名；若审批人反复要求查看导师信息，就应把该字段纳入申请详情。

图 4-4 原型反馈闭环



图注：本图说明原型设计应形成反馈闭环，让用户反馈进入需求修订和再次验证。

原型反馈前后对照

用户反馈	需求修订
学生看不懂“待确认”和“待审批”的区别	统一状态命名为“待审批”，并在详情页显示下一步处理人
审批人希望看到导师和实验用途	预约申请表增加导师、用途、预计人数，并在审批详情中展示
实验员担心设备维护时仍被预约	设备状态增加“维护中”，维护中设备不进入可预约时间段

学生不知道通知失败是否影响预约	预约结果页明确显示预约状态，通知失败只作为提示，不改变预约状态

4.9 质量属性需求：把“好用、安全、稳定”写具体

质量属性需求经常被写得过于抽象。应使用场景化方式描述：刺激源、刺激、环境、制品、响应和响应度量。

例如，安全性场景可以写为：普通学生在已登录环境下尝试访问审批接口，系统应拒绝访问并返回 403，同时记录安全日志。可用性场景可以写为：通知服务短时不可用时，预约记录仍应成功保存，通知进入重试队列。性能场景可以写为：学生查询一周内可预约时间段，常规数据量下 2 秒内返回。

这种写法让质量属性进入设计和测试，而不是停留在“系统要稳定”的口号。

4.10 项目实践：写一份轻量需求包

课程项目建议提交一份轻量需求包，至少包括以下内容。

模块	内容
利益相关者表	角色、诉求、痛点、冲突
典型用户	2-3 类核心用户画像
用例文本	3 个核心用例，含异常流程
需求规格	功能、数据、质量属性、约束
原型说明	关键页面和反馈记录
验收标准	每个核心需求如何验证

这份材料不追求篇幅，而追求可用于开发、测试和答辩说明。

本章与后续学习的关系

本章把用户声音转化为可验证规格。下一章将通过 C4、UML、ERD、DFD 等图示把需求转化为可沟通的系统模型。

4.11 本章小结

1. 需求不是用户第一句话，而是经过场景、用户、问题和规则分析后的系统约定。
2. 利益相关者分析能避免需求只满足单一视角。
3. 用例分析要写主流程、异常流程、前置条件、后置结果和业务规则。
4. 需求规格必须可验证，质量属性也应写成可测试场景。
5. 原型设计用于暴露理解偏差，并推动需求修订。

4.12 复习题与参考答案要点

【题型：名词解释｜难度：基础】基础概念题 1：什么是利益相关者?

参考答案要点：直接或间接受系统影响、参与使用、维护、监管或提供外部能力的人或组织。

【题型：名词解释｜难度：基础】基础概念题 2：用例描述应包含哪些要素？

参考答案要点：参与者、目标、前置条件、主成功场景、异常流程、后置结果和业务规则。

【题型：名词解释｜难度：基础】基础概念题 3：什么是可验证需求？

参考答案要点：能通过评审、测试或检查判断是否满足的需求，通常包含明确条件和度量标准。

【题型：简答题｜难度：中等】简答题 1：为什么不能把用户说的“更快”直接当作需求？

参考答案要点：更快可能指提交步骤少、反馈快、审批快、冲突少等不同目标，需要通过场景和问题分析澄清。

【题型：简答题｜难度：中等】简答题 2：说明原型设计在需求分析中的作用和风险。

参考答案要点：作用是低成本暴露理解偏差、验证流程和交互；风险是被误当成正式系统或只关注界面不关注规则。

【题型：案例分析｜难度：拔高】案例分析题：某预约系统需求写为“权限要安全”。请改写为可验证需求。

参考答案要点：按角色限制接口访问；未授权返回 403；审批操作记录日志；越权访问纳入测试用例；权限变更有审计记录。

【题型：项目应用｜难度：中等】项目实践题：为你的课程项目写一个核心用例文本。

参考答案要点：包含参与者、目标、前置条件、主流程、异常流程、后置结果和验收标准。

第 5 章 架构表达与系统建模

5.1 本章导读：图不是装饰，而是沟通结构

软件项目中的图示很容易走向两个极端：一种是没有图，所有设计都靠口头说明；另一种是画了很多图，但图只是在摆符号，无法解释系统边界、数据流、职责和约束。真正有价值的建模，是让不同角色能够围绕同一张图讨论问题。

实验室预约与设备管理系统至少需要回答几类问题：系统和校园环境的边界在哪里？有哪些外部系统？内部模块如何划分？核心业务对象是什么？预约从提交到审批经历哪些状态？并发冲突和通知失败在哪个环节处理？

本章介绍 C4 模型、UML 图、ERD、DFD 和流程建模的基本用途。重点不在于背图形符号，而在于选择合适图示表达合适问题，并让图示服务于需求、设计、开发和测试。

【你要记住】好图不是“画得复杂”，而是让边界、关系、职责和流动更清楚。

5.2 一个反面案例：图很多，但没人能按图开发

某项目组在设计文档中放了多张图：一个系统框图、一个数据库图、几个流程图和若干页面截图。看起来材料很完整，但评审时仍然回答不出关键问题：统一认证属于系统内部还是外部？预约冲突由前端、后端还是数据库处理？通知失败是否回滚预约？教师审批和实验员审批是同一流程还是两个流程？设备维护状态如何影响可预约状态？

问题不在于没有图，而在于图没有承担明确表达任务。系统框图没有边界，流程图没有角色泳道，数据库图没有说明业务规则，页面截图没有对应需求。图示变成了文档装饰。

改进方式是先确定每张图回答的问题。例如，C1 图回答系统处于什么环境；C2 图回答系统由哪些容器组成；C3 图回答后端内部模块职责；用例图回答参与者和目标；类图回答核心对象关系；状态图回答预约状态变化；DFD 回答数据如何流动。

5.3 建模的基本原则：对象、关系、边界、变化

任何建模活动都应关注四个方面。

对象是系统中被讨论的实体，例如学生、设备、预约、审批、通知、实验室。关系说明对象之间如何连接，例如一个学生可以提交多条预约，一台设备在同一时段只能有一个有效预约。边界说明系统内部和外部的分界，例如统一认证和邮件服务通常是外部依赖。变化说明状态如何转换，例如预约从待审批到已通过、已拒绝、已取消。

若图示无法表达这四类信息中的至少一类，就要重新判断它是否必要。

图 5-1 从分析到设计的建模链条

图注：本图说明建模链条从需求场景开始，逐步推导对象、数据、模块和测试依据。

5.3.1 从预约用例推导一组图

建模不是先选图形符号，而是从需求问题逐步推导表达方式。以“提交预约申请”为例，可以形成一条最小建模链：

用例分析
要回答的问题：谁使用系统，目标是什么
推荐图示：用例图/用例文本
实验室预约系统示例：学生提交预约，审批人处理申请

领域对象
要回答的问题：系统中有哪些核心概念
推荐图示：类图/领域对象表
实验室预约系统示例：学生、设备、预约、审批记录、通知任务

数据关系
要回答的问题：数据如何存储和约束
推荐图示：ERD
实验室预约系统示例：一台设备同一时段只能有一条有效预约

状态变化
要回答的问题：核心对象如何变化
推荐图示：状态图
实验室预约系统示例：草稿、待审批、已通过、已拒绝、已取消

协作流程
要回答的问题：多个角色如何完成业务
推荐图示：泳道图/活动图
实验室预约系统示例：学生提交、系统校验、教师审批、系统通知

系统结构
要回答的问题：系统边界和模块职责
推荐图示：C4 C1/C2/C3
实验室预约系统示例：Web、API、数据库、认证、通知服务

这条链路能避免“为了画图而画图”。每一张图都回答一个问题，并且能被后续开发、测试或评审使用。

提交预约申请的最小图例说明

- 用例图中至少包含学生、审批人、通知服务三个参与者，以及查看可用时段、提交预约、审批申请、发送通知四个用例。
- ERD 中至少包含 User、Device、Reservation、ApprovalRecord、NotificationTask 五个实体，并标明 Reservation 与 Device、User 的关系。
- 状态图中至少包含待审批、已通过、已拒绝、已取消四个状态，并标明“审批通过”“审批拒绝”“用户取消”等触发事件。
- C2 图中至少包含 Web 前端、后端 API、数据库、统一认证、邮件/通知服务。

【项目提示】课程项目的图不要求多，但每张图必须能回答一个具体问题。无法说明用途的图，应删除或重画。

5.4 C4 模型：从系统环境到代码组件

C4 模型用四个层次表达软件结构：Context、Container、Component、Code。课程项目通常重点使用前三层。

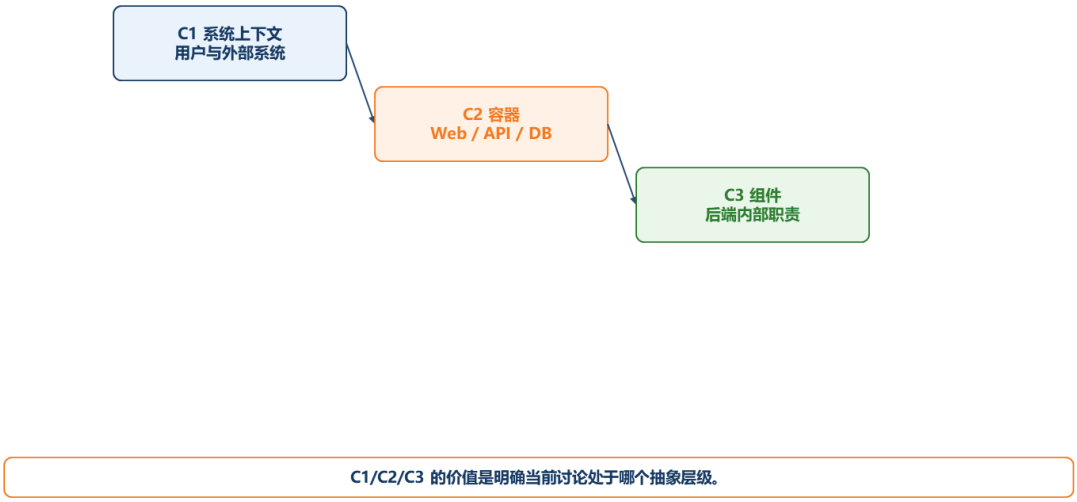
C1 系统上下文图说明系统与用户、外部系统的关系。实验室预约系统的外部对象包括学生、教师、实验员、学院管理员、统一认证、邮件服务和可能的门禁系统。

C2 容器图说明系统由哪些可运行或可部署单元组成，例如 Web 前端、后端 API、数据库、缓存、文件存储、消息队列和外部服务。C2 图回答的是“系统运行时由哪些部分协作”。

C3 组件图说明某个容器内部如何划分职责。例如后端 API 内部可以划分为预约服务、设备服务、审批服务、通知适配、权限模块和数据访问层。C3 图不应重复 C2 的部署结构，而应解释内部职责。

【易错点】C1、C2、C3 的层次不能混用。把数据库、用户、Controller 和按钮画在同一张图里，会让图既看不清边界，也看不清职责。

图 5-2 实验室预约系统 C1/C2/C3 层次



图注：本图说明 C4 模型的价值在于区分抽象层次，避免把用户、数据库和代码组件混在一张图里。

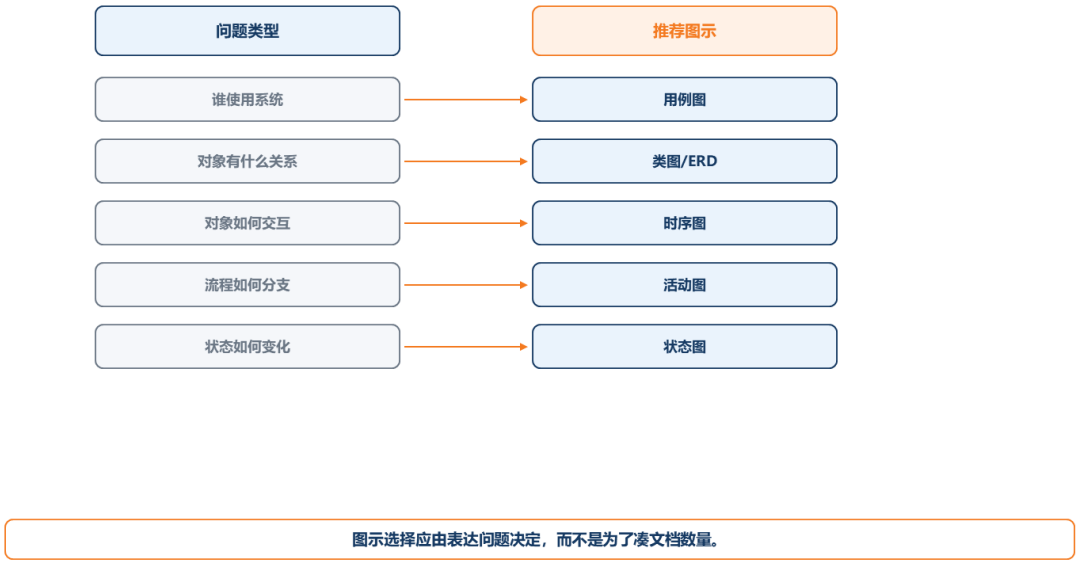
5.5 UML 图：根据问题选择图

UML 不是必须全部使用的符号库。课程项目常用图包括用例图、类图、时序图、活动图和状态图。

用例图适合表达参与者和系统目标；类图适合表达领域对象及关系；时序图适合表达对象之间按时间顺序的交互；活动图适合表达流程分支；状态图适合表达对象生命周期。

在实验室预约系统中，“提交预约申请”可以用用例图说明参与者，用活动图说明流程分支，用时序图说明前端、后端、数据库和通知服务之间的调用，用状态图说明预约状态变化，用类图说明学生、设备、预约、审批记录之间的关系。

图 5-3 UML 图选择矩阵



图注：本图用问题类型选择 UML 图，帮助项目组判断当前应该画哪一种图。

UML 最小示例：提交预约申请

图	最小内容	常见错误
用例图	学生、审批人、通知服务；提交预约、审批申请、发送通知	把页面按钮画成用例
时序图	前端调用后端，后端校验冲突，写数据库，生成通知任务	只画模块框，不画消息顺序
活动图	选择设备、校验冲突、提交、审批、通知，包含异常分支	把所有流程画成单线直走
状态图	待审批、已通过、已拒绝、已取消、已过期	把用户操作当成状态

【易错点】UML 图的价值不是符号完整，而是让业务规则、对象关系和状态变化可讨论。符号可以简化，关系不能含糊。

5.6 ERD 与 DFD：数据结构与数据流动

ERD（实体关系图）用于表达数据实体、属性和关系。实验室预约系统中的实体包括用户、角色、设备、实验室、预约、审批记录、通知记录和维护记录。ERD 可以帮助团队检查数据是否支持业务规则，例如一台设备在同一时间段不能有多个有效预约。

DFD（数据流图）用于表达数据在处理过程、外部实体和数据存储之间如何流动。它适合说明“数据从哪里来、经过哪些处理、存到哪里、输出给谁”。例如预约申请从学生输入进入系统，经过冲突校验和规则判断，生成预约记录和审批任务，再通知审批人。

ERD 关注“数据是什么结构”，DFD 关注“数据如何流动”。二者不能互相替代。

【考试提示】比较 ERD 和 DFD 时，应说明一个表达静态数据关系，一个表达动态数据流动，并结合具体业务说明。

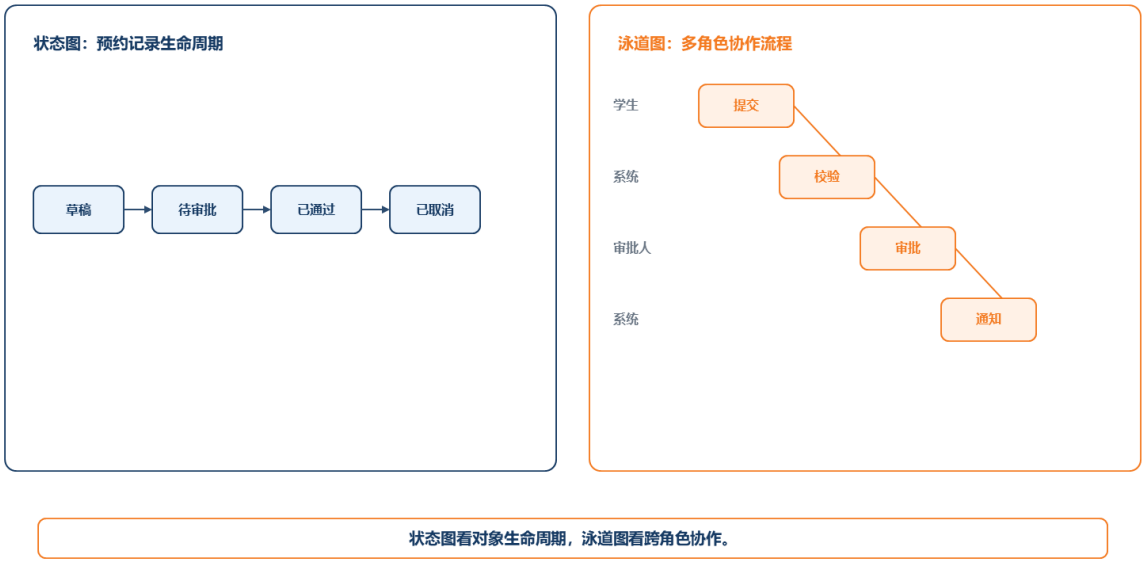
5.7 状态与流程：不要把审批流程画成一条直线

预约业务不是简单的提交、审批、完成。真实系统需要处理待审批、已通过、已拒绝、已取消、已过期、设备维护中、通知失败等状态或异常。

状态图适合描述一个对象的生命周期。例如预约记录从“草稿”进入“待审批”，可能变成“已通过”或“已拒绝”；已通过后可能被取消；过期后进入历史状态。每个状态转换都应有触发事件和业务规则。

活动图或泳道图适合描述多个角色协作。例如学生提交，系统校验，教师审批，实验员确认设备状态，系统发送通知。泳道能清楚说明谁做动作、系统如何响应、规则在哪一步生效。

图 5-4 预约状态图与泳道流程图对照



图注：本图对比状态图和泳道流程图：一个说明预约记录如何变化，一个说明多角色如何协作完成流程。

5.8 好图的检查清单

一张图是否有效，可以用以下清单检查。

检查项	判断问题
标题明确	这张图回答什么问题
层级一致	是否混合了不同抽象层
边界清楚	系统内外是否分明
关系清楚	连接线是否有方向和含义
术语一致	名称是否与需求和代码一致
可验证	能否推导测试或开发任务

图示不是越细越好。若一张图太复杂，应拆成不同层级或不同问题的图。若一张图无法帮助读者做判断，应删除或重画。

5.9 项目实践：提交一组最小建模图

课程项目建议提交以下最小图集。

图示	目的
C1 上下文图	说明系统边界、用户和外部依赖
C2 容器图	说明前端、后端、数据库和外部服务
C3 核心模块图	说明后端主要职责划分
用例图	说明主要参与者和目标
类图/ERD	说明核心数据对象关系
状态图或泳道图	说明关键流程和状态变化

每张图后面应配一段说明：这张图回答什么问题，哪些设计决策由它支撑，哪些测试可以从它推导。

建模图质量评分表

维度	合格标准
问题明确	图题能说明本图回答什么问题
层级一致	不把用户、按钮、数据库、类和服务混在同一层
边界清楚	系统内部、外部系统和用户角色有明确分界
关系可读	连接线有方向和含义，避免交叉混乱
术语一致	图中的名称与需求文档、代码模块、数据库表一致
可用于后续	能支撑开发任务、测试用例或架构决策

本章与后续学习的关系

本章解决如何表达系统结构。下一章将进一步讨论为什么这样组织结构，以及如何选择架构风格和关键决策。

5.10 本章小结

1. 建模的目标是表达边界、关系、职责和变化。
2. C4 模型帮助团队按抽象层级表达系统结构。
3. UML 图应按问题选择，不能为了凑数量而画图。
4. ERD 表达数据结构，DFD 表达数据流动。
5. 好图必须能支撑沟通、开发、测试或评审。

5.11 复习题与参考答案要点

【题型：名词解释｜难度：基础】基础概念题 1：C4 模型的前三层分别是什么？

参考答案要点：Context 系统上下文、Container 容器、Component 组件；分别表达系统环境、运行单元和内部职责。

【题型：名词解释｜难度：基础】基础概念题 2：ERD 和 DFD 的区别是什么？

参考答案要点：ERD 表达实体、属性和关系；DFD 表达数据在外部实体、处理过程和数据存储之间的流动。

【题型：名词解释｜难度：基础】基础概念题 3：状态图适合表达什么问题？

参考答案要点：适合表达对象生命周期、状态转换、触发事件和状态约束。

【题型：简答题｜难度：中等】简答题 1：为什么说图示不是装饰？

参考答案要点：图示应帮助说明边界、职责、数据、流程和状态，并支撑沟通、开发、测试和评审。

【题型：简答题｜难度：中等】简答题 2：实验室预约系统至少需要哪些图？各自回答什么问题？

参考答案要点：C1/C2/C3 说明结构层级，用例图说明参与者目标，ERD 说明数据关系，状态图或泳道图说明流程和状态变化。

【题型：案例分析｜难度：拔高】案例分析题：某系统图把用户、数据库、按钮、Controller、外部服务画在同一层。请指出问题并改进。

参考答案要点：抽象层级混乱、边界不清、职责不明；应拆成 C1 上下文、C2 容器、C3 组件和必要的流程/状态图。

【题型：项目应用｜难度：中等】项目实践题：为课程项目绘制一张 C2 容器图并说明外部依赖。

参考答案要点：标出前端、后端、数据库、缓存/文件存储、外部认证/通知服务，并说明数据和调用关系。

第 6 章 架构设计、架构风格与分布式系统

6.1 本章导读：为什么功能能跑不代表架构合理

软件系统的危险之处在于：很多设计问题在第一天并不会暴露。页面能打开，按钮能点击，数据库能保存记录，用户看起来也能完成操作。但当系统开始被多人同时使用，开始接入外部服务，开始增加新角色和新规则，原来“能跑”的设计就可能迅速变成“没人敢改”的系统。

以“实验室预约与设备管理系统”为例，首版功能并不复杂：学生查看设备并提交预约，实验员审批，教师查看安排，系统发送通知。如果只看功能清单，这似乎就是几个页面、几张表和几个接口。但是系统一旦进入真实使用，就会遇到架构问题。

例如，两名学生同时预约同一台设备、同一时间段，系统能否保证只产生一条有效预约？邮件服务临时不可用时，预约是否应该失败？普通学生能否通过修改接口参数访问管理员功能？后续如果增加“学院级设备共享”“跨实验室借用”“设备维修状态联动”，系统是否还能在不大改的情况下演进？

这些问题不是简单的编码问题，而是架构决策问题。架构设计关心的不是“代码能不能写出来”，而是系统如何组织，边界如何划分，哪些链路必须强一致，哪些能力可以异步处理，哪些风险要被记录，哪些技术债可以暂时接受。

本章的目标是帮助学生建立一个基本判断：架构不是技术栈清单，而是在约束条件下对系统结构做出的关键决策。

【你要记住】

本章最重要的判断不是“哪种架构最先进”，而是“在当前约束下，哪种结构决策更能控制复杂性，并且能说明取舍理由”。

6.2 一个反面案例：实验室预约系统的“快写版”设计

6.2.1 前因：为了尽快完成首版

某小组要在较短时间内完成实验室预约系统。为了尽快看到效果，团队采用了最直接的写法：

- 所有业务逻辑都写在 Controller 中。
- 预约冲突主要靠前端判断。
- 预约成功后同步调用邮件接口发送通知。
- 权限主要靠前端隐藏按钮控制。
- 数据表能满足当前页面即可，没有明确模块边界。
- 没有记录任何架构决策，也没有说明为什么这样设计。

短期看，这种设计很有吸引力。页面很快做出来，接口也能跑通，演示时学生可以提交预约，实验员可以审批，系统也能发送通知。问题在于，这种“快写版”只是功能跑通，并没有真正控制复杂性。

6.2.2 冲突：真实使用后问题集中暴露

当系统进入真实场景后，问题开始出现。

第一，预约冲突无法可靠避免。前端判断设备是否空闲，只能看到页面加载时的状态。两个学生几乎同时提交预约时，前端都认为该时段可用，后端如果没有事务、唯一约束或锁机制，就可能产生两条有效预约。

第二，通知失败影响主流程。预约已经写入数据库，但邮件服务超时，接口等待很久后返回失败。用户看到“预约失败”，但后台可能已经保存了预约记录，造成状态不一致。

第三，权限边界不可靠。前端隐藏了“审批”按钮，但如果学生直接调用审批接口，后端没有统一权限校验，就可能出现越权操作。

第四，维护成本快速上升。所有逻辑堆在 Controller 中，预约、审批、通知、权限和设备状态互相交织。新增“导师确认”规则时，开发者不知道应该改哪里，也不确定改完会影响哪些流程。

6.2.3 后果：系统从“能跑”变成“难改”

这种设计的最大风险不是首版无法演示，而是后续无法稳定演进。系统会出现几类后果：

- 缺陷定位困难：出问题时无法判断是预约规则、审批规则、通知服务还是权限控制导致。
- 修改影响范围不可控：改一个审批条件可能影响预约状态、通知发送和统计报表。
- 测试困难：业务逻辑和外部依赖耦合在一起，无法单独测试核心规则。
- 安全风险上升：权限散落在页面和接口中，容易遗漏。
- 团队协作变慢：每个人都怕改到别人负责的逻辑。

6.2.4 改法：用架构决策替代临时拼接

改进不是简单地换一个框架，而是重新明确结构决策：

- 预约冲突检测放在后端核心业务规则中，并通过数据库约束、事务或锁机制保障一致性。
- 通知从主流程中拆出，预约成功后记录通知任务，邮件发送异步执行，失败可重试。
- 权限模型统一放在后端，前端隐藏按钮只作为体验优化，不作为安全边界。
- 后端按业务职责划分模块，例如用户权限、设备、预约、审批、通知、统计。
- 对关键选择写 ADR，说明为什么首版采用模块化单体，为什么通知异步，为什么权限统一后端校验。

这个案例说明：架构设计不是在功能之后“画一张漂亮图”，而是在系统还小的时候就把关键边界和取舍说清楚。

这个反面案例可以概括为一条推演链：错误方案是把功能快速堆到一起；问题暴露在并发、外部依赖、权限和维护变化中；架构判断要识别哪些链路必须强一致、哪些能力可以异步、哪些边界必须统一；改进方案则是用模块化单体、后端一致性控制、异步通知、统一权限模型和 ADR 记录来控制复杂性。

【易错点】

“首版能演示”不等于“架构合理”。演示通常只覆盖正常路径，真实系统会暴露并发、失败、越权和后续变化问题。

6.3 架构设计到底在决策什么

6.3.1 架构是关键结构决策的集合

架构设计关注的是那些影响范围大、回退代价高、长期影响系统演进的选择。它回答的问题包括：

- 系统由哪些主要部分组成？
- 各部分之间如何协作？
- 数据如何流动和保持一致？
- 外部依赖失败时系统如何处理？
- 质量属性通过什么结构得到保障？
- 后续变化发生时，哪些部分可以稳定，哪些部分可以替换？

对实验室预约系统来说，以下选择属于架构决策：

- 首版采用模块化单体，而不是微服务。
- 预约冲突检测必须由后端保障强一致。
- 通知采用异步任务，不阻塞预约主流程。
- 权限校验统一在后端完成。
- 外部统一认证系统通过适配层接入，避免业务核心直接依赖具体接口。

这些决策会影响模块划分、数据设计、测试方式、部署方式和后续演进，因此属于架构层面的选择。

6.3.2 什么不是关键决策

并不是所有技术选择都是架构决策。以下内容通常不是架构重点：

- 按钮颜色、页面布局等纯界面细节。
- 普通变量命名、局部函数写法。
- 不影响系统结构的小工具库选择。
- 可以低成本替换的局部实现细节。

当然，如果某个工具选择会影响系统结构、团队协作或长期演进，它也可能上升为架构决策。例如，是否引入消息队列就不只是工具选择，因为它会改变同步调用关系、失败处理方式和测试策略。

6.3.3 技术栈与架构理由的区别

“Vue + Spring Boot + MySQL + Redis”不是架构方案，只是技术栈清单。技术栈回答“用了什么”，架构方案回答“为什么这样组织系统”。

【考试提示】

如果题目要求说明“为什么技术栈清单不能代替架构方案”，答题时至少要写出：技术栈只说明工具，架构方案还要说明模块边界、协作方式、质量属性、风险和取舍理由。

一个更合格的表达应类似这样：

- 前端只负责交互和状态展示，不承担预约冲突的最终判断。
- 后端按预约、设备、审批、通知、权限划分模块。
- 预约冲突由数据库唯一约束和业务事务共同保障。
- 通知从主流程中异步拆出，失败记录并重试。
- Redis 只用于缓存设备展示信息，不作为预约可用性的最终判断依据。

这里的重点不是用了哪些技术，而是说明了责任边界、一致性策略和故障处理方式。

6.3.4 Redis 例子：工具名不是架构，取舍才是架构

如果方案中只写“使用 Redis 提升性能”，这仍然不够。架构层面至少要追问：

- 缓存什么数据？设备列表、设备详情，还是预约时段？
- 缓存何时失效？审批通过、设备维修、预约取消时是否更新？
- 缓存和数据库不一致时，以谁为准？
- 缓存故障时，系统是降级、回源数据库，还是直接失败？
- 缓存是否参与预约冲突判断？

对于实验室预约系统，设备展示信息可以缓存，因为短时间不一致通常可以接受；但预约占用状态不应只依赖缓存，因为错误的可用状态会直接导致冲突预约。这个判断才是架构问题。

【易错点】

Redis 不是架构理由。“缓存什么、何时失效、与数据库不一致时以谁为准、缓存故障时如何降级”，这些才是架构设计需要回答的问题。

6.4 质量属性如何驱动架构选择

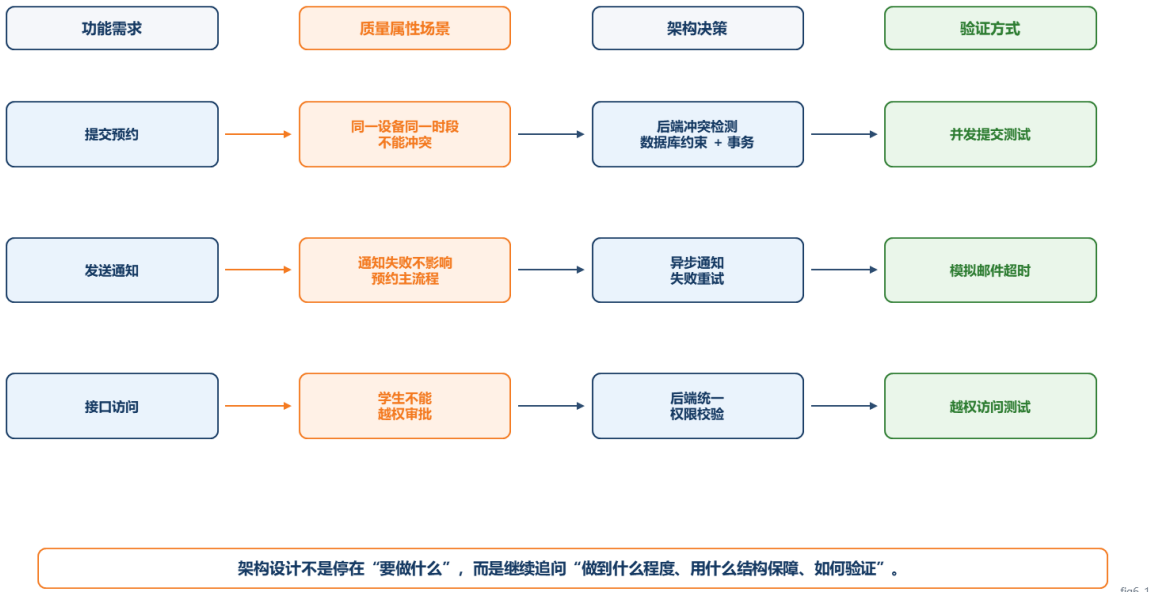
6.4.1 功能需求与质量属性的区别

功能需求说明系统要做什么，质量属性说明系统要以什么质量完成这些功能。

类型	示例	对架构的影响
功能需求	学生可以提交预约申请	需要预约提交接口、页面和数据表
一致性要求	同一设备同一时段不能出现两条有效预约	影响事务、锁、唯一约束和并发控制
可用性要求	通知服务失败不能导致预约失败	影响同步/异步拆分和失败重试机制
安全性要求	学生不能调用实验员审批接口	影响统一认证、授权和接口权限校验
可维护性要求	新增审批规则时不应改动大量无关代码	影响模块边界、业务规则位置和依赖方向

架构设计最重要的工作之一，就是把“系统要稳定、安全、可维护”这类泛泛表述，转化为具体的质量场景和结构决策。

图 6-1 质量属性如何驱动架构决策



图注：本图说明功能需求需要进一步转化为质量属性场景，再落到架构决策和验证方式。

6.4.2 强一致如何影响数据库和事务设计

预约占用属于核心业务正确性。若系统允许同一设备同一时段出现两个有效预约，即使之后可以人工修正，也已经破坏了用户信任。因此预约占用不能只靠前端判断，也不适合简单采用最终一致。

合理做法包括：

- 后端统一检查预约时段是否可用。
- 数据库层增加唯一约束或排他约束。
- 预约提交过程使用事务保证状态变化原子性。
- 并发高时考虑锁机制或基于版本号的并发控制。

这里的架构判断是：预约结果必须强一致，通知结果可以延迟。

6.4.3 可用性如何影响异步通知

通知很重要，但它不应阻塞预约主流程。若邮件服务故障导致预约失败，用户会遇到不合理体验：明明设备时段已占用，却因为邮件服务不可用而无法完成预约。

更合理的做法是：

- 预约成功后先记录通知任务。
- 后台异步发送邮件或站内消息。
- 通知失败时记录错误并重试。

- 管理端可以查看通知失败记录。

这样做牺牲的是通知的即时性，换来的是预约主流程的可用性。

6.4.4 安全性如何影响统一权限模型

权限不能只依赖前端。前端隐藏按钮只能减少误操作，不能阻止恶意或误用接口。系统必须在后端统一判断用户身份、角色和资源权限。

例如：

- 学生只能查看和取消自己的预约。
- 实验员只能审批自己负责实验室的预约。
- 指导教师只能查看与自己指导相关的实验安排。
- 学院管理员可以配置规则，但不能绕过审计。

这些规则如果分散在多个 Controller 中，后续新增角色时很容易遗漏。统一权限模型能提升安全性，也能提升可维护性。

6.4.5 可维护性如何影响模块边界

可维护性不是“代码写得整洁”这么简单。它要求系统的变化能被限制在合理范围内。

实验室预约系统至少应划分以下业务模块：

- 用户与权限模块：负责身份、角色和授权。
- 设备模块：负责设备信息、状态和维护。
- 预约模块：负责预约申请、取消、冲突检测。
- 审批模块：负责审批流程和规则。
- 通知模块：负责消息生成、发送、失败重试。
- 统计模块：负责使用率、预约量和报表。

模块边界清楚时，新增“设备维修期间禁止预约”规则，主要影响设备和预约模块；新增“审批结果通知导师”，主要影响审批和通知模块。边界不清时，任何变化都可能牵动全系统。

6.5 ADR：把架构选择写成可审查的理由

6.5.1 为什么需要 ADR

架构决策如果只停留在口头，很快就会丢失上下文。几周后团队可能只记得“当时选了模块化单体”，却忘了为什么没有选微服务；几个月后新成员加入，只能从代码中猜测设计意图。

ADR 的作用是保留决策背景和取舍过程。它不需要很长，但要能回答三个问题：

- 当时面对什么约束？
- 为什么选这个方案？
- 什么情况下需要重新评估？

6.5.2 完整 ADR 示例

ADR-001：首版采用模块化单体

背景

内容：实验室预约系统首版由 5 人小组在 6 周内完成，业务边界尚在验证中，部署环境简单，团队缺少复杂运维经验。系统需要支持预约、审批、设备管理、通知和权限控制。

决策

内容：首版采用模块化单体。系统统一部署为一个后端应用，内部按用户权限、设备、预约、审批、通知、统计划分模块。模块之间通过清晰接口协作，避免任意跨模块访问数据表。

放弃方案

内容：暂不采用微服务。原因是服务拆分会引入独立部署、服务发现、接口治理、分布式事务、链路追踪等成本，超出首版团队能力和项目周期。也不采用无边界的简单单体，因为它会导致后续难以维护。

理由

内容：模块化单体能降低部署和调试复杂度，适合小团队首版交付；同时通过内部模块边界保留后续演进空间。

风险

内容：如果模块边界执行不严格，系统仍可能退化为“大泥球”；如果后续用户规模显著扩大，单体部署可能限制独立扩展。

观察指标

内容：模块间循环依赖数量；新增需求修改影响范围；部署耗时；缺陷定位时间；是否出现某个模块需要独立扩展的压力。

6.5.3 逐项解释

背景说明决策面对的条件。没有背景，决策就像脱离现场的结论。例如“采用模块化单体”本身没有绝对正确，只有在“5 人团队、6 周首版、运维能力有限、业务边界尚未稳定”这个背景下才更合理。

决策要写清楚最终选择，而不是只写技术名词。这里不是简单写“采用 Spring Boot”，而是说明统一部署、内部模块划分和模块协作原则。

放弃方案最能体现架构判断。只写“选了模块化单体”无法证明经过权衡；写清“为什么暂不采用微服务，为什么也不接受无边界简单单体”，才能让别人审查这个决策是否合理。

【你要记住】

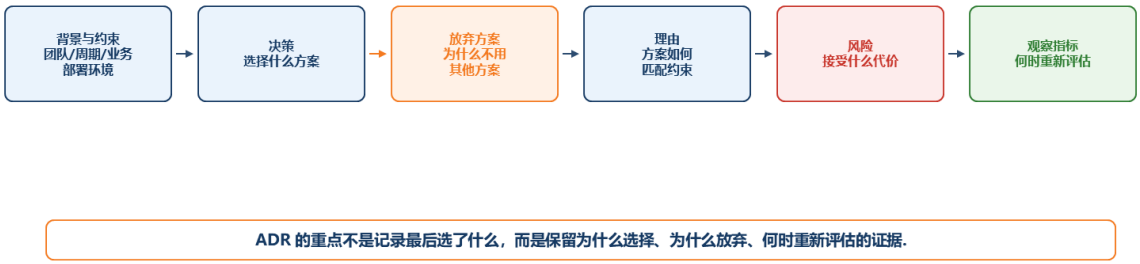
ADR 中最容易被漏掉、也最能体现专业判断的是“放弃方案”。它说明团队不是只写了一个结论，而是真的比较过其他可能性。

理由连接背景和决策，说明为什么该方案匹配当前约束。

风险承认当前方案不是完美方案。架构决策不是寻找没有缺点的方案，而是在已知条件下接受可管理的风险。

观察指标用于后续复盘。如果未来模块间依赖越来越乱，部署越来越慢，缺陷定位越来越困难，就说明原有决策需要调整。

图 6-3 ADR 结构示意图



图注：本图说明 ADR 是一条可审查的架构推理链，重点是保留选择、放弃和重新评估的理由。

6.6 架构风格不是名词表，而是问题匹配

架构风格不应被当作名词背诵。判断一种风格是否合适，应先看它解决什么问题，再看它引入什么成本，最后看团队是否具备使用它的条件。

6.6.1 结构组织类：分层、模块化单体、六边形架构

这类风格主要解决“系统内部如何组织”的问题。

分层架构

解决的问题：区分表现层、业务层、数据访问层，避免页面直接操作数据库

适用条件：业务复杂度不高，需要建立基本工程结构

引入成本：若业务规则散落在各层，可能形成贫血模型

课程项目建议：推荐作为基础结构

模块化单体
解决的问题：在一个部署单元内保持清晰业务边界
适用条件：小团队、首版交付、业务边界还在探索
引入成本：需要自律维护模块边界
课程项目建议：强烈推荐

六边形架构
解决的问题：隔离业务核心与外部技术依赖
适用条件：业务规则重要，外部接口可能变化，测试要求高
引入成本：抽象成本较高，小项目容易过度设计
课程项目建议：可在关键模块局部使用

对于实验室预约系统，推荐采用“分层 + 模块化单体”。核心预约规则可以更接近六边形架构思想：业务规则不要直接依赖邮件接口、统一认证接口或具体数据库细节。

6.6.2 扩展复用类：组件化、插件化、DDD 限界上下文

这类风格主要解决“能力如何复用，变化如何隔离”的问题。

组件化
解决的问题：把可复用功能封装为组件
适用条件：多处使用同类功能，如表格、审批状态、通知模板
引入成本：需要稳定接口和版本管理
课程项目建议：推荐在前端或公共能力中使用

插件化
解决的问题：让系统通过扩展点接入新能力
适用条件：业务扩展点明确，如多种通知渠道、多种报表导出
引入成本：设计扩展点有成本，过早插件化会变复杂
课程项目建议：谨慎使用，只在明确扩展点使用

DDD 限界上下文
解决的问题：用业务边界控制模型含义
适用条件：业务概念复杂，不同部门对同一术语含义不同
引入成本：学习成本较高，需要深入业务建模
课程项目建议：可借鉴“边界”思想，不必完整套用

例如“预约”在学生视角是申请，在实验员视角是审批任务，在统计视角是设备使用记录。若系统规模扩大，需要更认真地区分这些上下文；但在课程项目首版中，不必把 DDD 术语全部搬进方案，只需要把业务边界说清。

6.6.3 分布运行类：SOA、微服务、事件驱动

这类风格主要解决“系统如何跨进程、跨服务、跨组织协作”的问题。

SOA
解决的问题：多系统共享服务能力
适用条件：组织内已有多个系统，需要复用认证、报表、设备数据等能力
引入成本：服务治理和接口管理复杂
课程项目建议：可理解，不建议首版完整实现

微服务
解决的问题：独立部署、独立扩展、团队自治
适用条件：业务边界清晰，团队规模较大，具备 DevOps、监控和自动化测试能力
引入成本：分布式事务、链路追踪、部署治理成本高
课程项目建议：不建议小团队首版默认采用

事件驱动
解决的问题：异步解耦、削峰、状态传播
适用条件：通知、审计、统计、异步任务等场景明显
引入成本：调试复杂，需要处理幂等、顺序和失败重试
课程项目建议：可局部用于通知和审计

对实验室预约系统，通知、审计和统计适合借鉴事件驱动思想；预约占用、权限校验等核心链路不宜轻易异步化。

6.6.4 四组快速对比

简单单体 vs 模块化单体

对比项	简单单体	模块化单体
内部边界	不清晰，逻辑容易混在一起	按业务职责划分模块
部署方式	一个应用	一个应用
维护成本	随规模上升很快变高	通过模块边界控制变化
课程项目建议	不推荐	推荐

模块化单体 vs 微服务

对比项	模块化单体	微服务

部署复杂度	低	高
适用团队	小团队、首版项目	较大团队、边界清晰
数据一致性	相对容易处理	需要处理分布式一致性
课程项目建议	首选	不作为默认答案

微服务 vs 分布式单体

对比项	真正微服务	分布式单体
服务边界	清晰，围绕业务能力	表面拆开，实际强耦合
数据归属	明确	常共享数据库
发布方式	可独立发布	经常必须一起发布
判断结论	有自治能力	复杂度上升但收益不足

同步调用 vs 异步事件

对比项	同步调用	异步事件
适合场景	需要立即结果和强一致	通知、审计、统计、削峰
优点	流程直接，结果明确	解耦主流程，提高可用性
风险	外部依赖会阻塞主流程	调试、幂等、重试更复杂
预约系统示例	提交预约、权限判断	通知发送、统计更新

【项目提示】

课程项目首版不要为了“显得高级”而默认写微服务。若团队不能说明服务边界、数据归属、独立部署和故障处理方式，优先采用模块化单体。

6.7 模块化单体、微服务与分布式单体

6.7.1 为什么小团队首版更适合模块化单体

课程项目通常具有几个特点：团队人数少，周期短，业务边界还在探索，部署和运维能力有限。在这种条件下，模块化单体通常比微服务更合适。

模块化单体的优势是：

- 一个应用部署，环境简单。
- 本地调试和测试成本低。
- 事务和一致性处理更直接。
- 通过内部模块边界保留演进空间。

它并不等于“随便写成一个大单体”。真正的模块化单体要求内部边界清晰、依赖方向受控、业务规则有明确归属。

6.7.2 什么是真正的模块化单体

真正的模块化单体至少应满足：

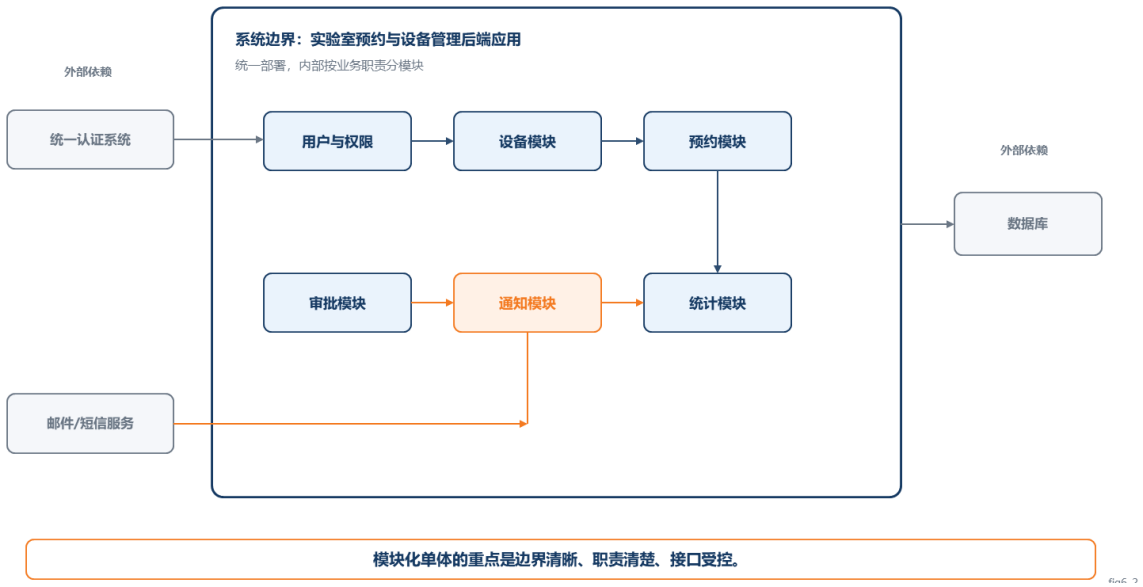
- 按业务职责划分模块，而不是按个人分工随意划分。
- 模块之间通过接口协作，不随意访问彼此内部数据。
- 核心业务规则不散落在 Controller 或页面脚本中。
- 模块依赖方向清楚，避免循环依赖。
- 保留后续拆分可能性，但不为未来不确定需求过度设计。

实验室预约系统中，预约模块可以调用设备模块查询设备状态，可以调用通知模块创建通知任务，但不应直接修改通知模块内部表结构。审批模块可以改变预约状态，但状态流转规则应有统一位置维护。

【你要记住】

模块化单体不是“一个应用随便写”。它的关键是内部模块边界清晰、依赖方向受控、业务规则有明确归属。

图 6-2 实验室预约系统模块化单体结构



图注：本图说明模块化单体可以统一部署，但内部仍要保持清晰的业务边界和受控依赖。

6.7.3 什么情况下可以考虑微服务

可以考虑微服务的条件包括：

- 业务边界已经稳定，不同服务有清晰职责。
- 团队规模足够，每个服务能由相对独立的小组负责。
- 系统需要独立部署、独立扩展或故障隔离。

- 团队具备自动化测试、持续交付、监控告警和故障排查能力。
- 能接受分布式一致性、接口版本管理和链路追踪带来的成本。

如果只是为了“看起来先进”而拆服务，往往得不到微服务收益，只会提前承担分布式复杂性。

【易错点】

微服务不是默认答案。只有业务边界清晰、团队具备独立部署和运维能力、服务确实需要独立扩展时，微服务才可能带来收益。

6.7.4 什么是分布式单体

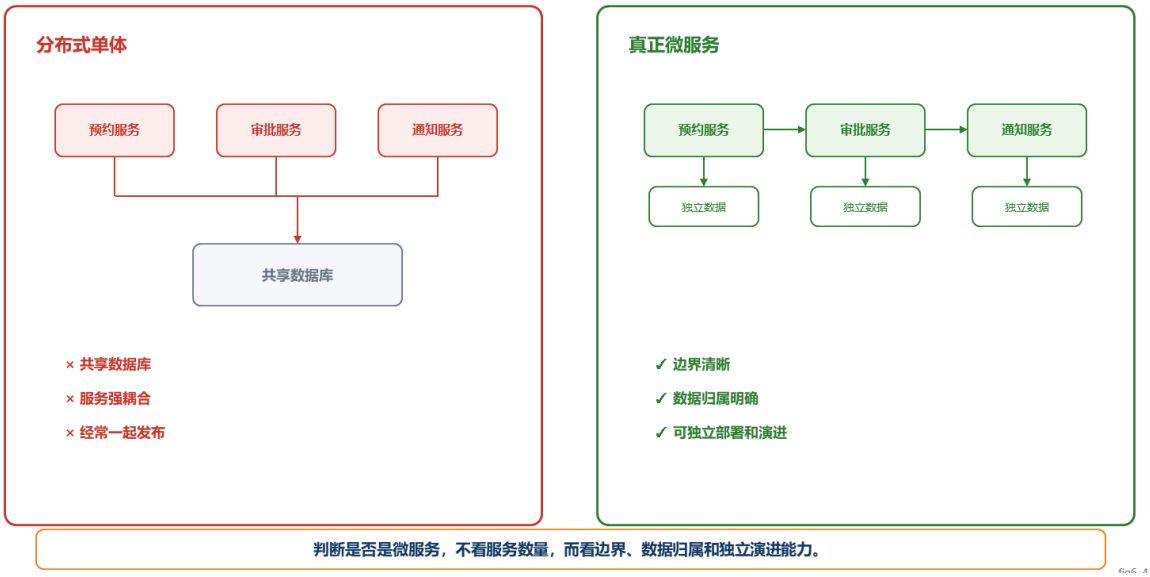
分布式单体是最常见的错误形态：系统被拆成多个服务，但服务之间高度耦合，必须一起发布、一起修改、一起回滚。它既失去了单体的简单性，又没有获得微服务的独立性。

典型表现包括：

- 服务之间互相调用链很深，任何小改动都牵动多个服务。
- 数据库表仍然被多个服务共享和直接修改。
- 一个服务无法独立测试，必须启动全部服务。
- 发布一个小功能需要多个服务同时上线。
- 接口没有版本治理，调用方和提供方互相卡住。

“拆成多个服务”不等于微服务。真正的微服务要求业务边界清晰、数据归属明确、服务能独立部署和独立演进。

图 6-4 分布式单体 vs 真正微服务



图注：本图对比共享数据库、强耦合的分布式单体与边界清晰、可独立演进的微服务。

6.7.5 实验室预约系统对比

简单单体
做法：所有逻辑堆在一个应用中，模块边界不清
短期效果：开发最快
长期风险：难维护、难测试、难演进
判断：不推荐

模块化单体
做法：一个应用部署，内部按业务模块划分
短期效果：交付稳定，成本可控
长期风险：需要持续维护边界
判断：首版推荐

分布式单体
做法：拆成多个服务，但共享数据库、一起发布
短期效果：看似现代
长期风险：调试、部署、一致性成本都上升
判断：应避免

真正微服务
做法：服务自治、数据归属明确、独立部署
短期效果：适合较大规模
长期风险：对工程能力要求高
判断：后续演进可考虑

6.8 同步、异步与一致性取舍

6.8.1 预约占用为什么不能最终一致

预约占用直接决定业务正确性。若两个用户同时预约同一设备，系统必须立即给出唯一结果。不能先让两人都成功，再通过异步任务修正其中一个。因为用户已经基于系统结果安排实验时间，事后修正会造成实际损失。

因此，预约占用链路应强一致：

- 后端统一执行冲突检测。
- 数据库层提供唯一约束或排他控制。
- 状态更新在事务中完成。

- 并发提交时必须有确定结果。

6.8.2 通知为什么可以异步

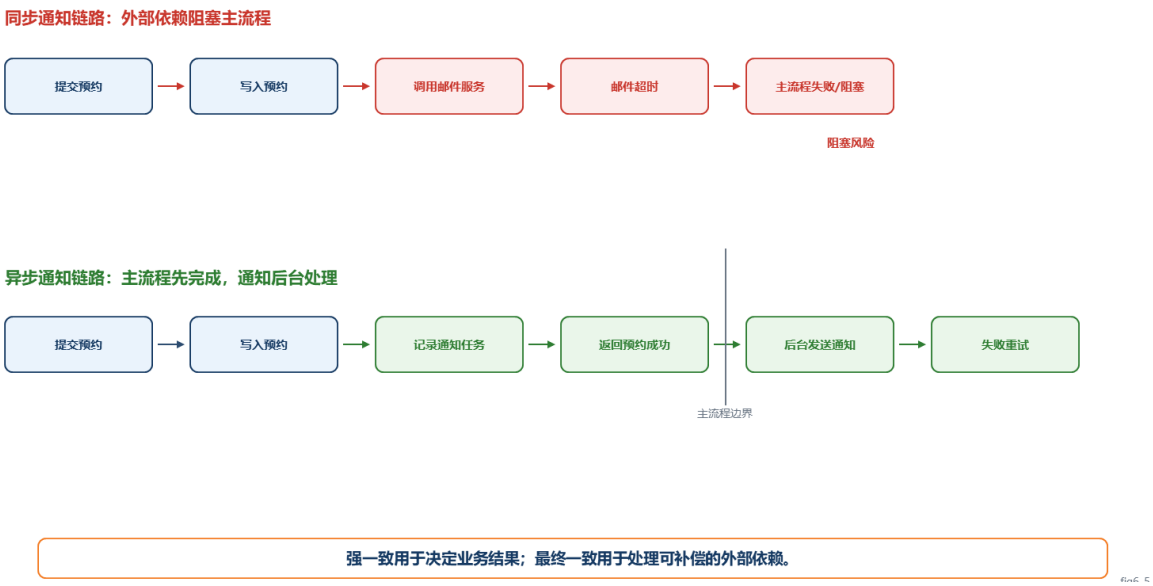
通知与预约结果相关，但它不是预约是否成立的最终依据。邮件晚到几分钟通常可以接受，邮件失败后也可以重试或通过站内消息补偿。

因此，通知适合异步处理：

- 预约成功后生成通知任务。
- 后台任务或消息队列负责发送。
- 发送失败记录原因并重试。
- 管理端可查看失败通知。

这种设计将“预约是否成功”和“通知是否送达”分开，避免外部服务故障拖垮核心业务。

图 6-5 同步通知 vs 异步通知链路对比



图注：本图说明预约占用属于主业务结果，应保持强一致；通知送达属于外部依赖，可异步处理并最终一致。

6.8.3 哪些链路必须强一致

通常应强一致的链路包括：

- 设备时段占用。
- 预约状态流转。
- 权限授权结果。
- 付款或扣费类操作。
- 影响业务唯一性的关键记录。

这些链路一旦错误，系统会产生直接业务损害。

6.8.4 哪些链路可以最终一致

通常可以最终一致的链路包括：

- 邮件、短信、站内通知。
- 使用率统计。
- 操作审计归档。
- 搜索索引更新。
- 报表汇总。

这些能力重要，但短暂延迟通常可接受。架构设计应把它们从主流程中拆出，减少同步依赖。

6.8.5 质量属性之间的取舍

架构设计不是让所有质量属性同时最大化，而是在冲突中排序。

取舍	示例	判断
一致性 vs 性能	预约占用需要强一致，可能增加锁和事务成本	预约正确性优先
可用性 vs 即时性	通知异步后不保证立即送达	主流程可用性优先
安全性 vs 开发速度	后端统一权限增加开发工作	安全边界优先
可维护性 vs 短期速度	模块边界设计增加前期成本	长期可维护性优先

6.8.6 业务动作与一致性要求对照表

提交预约
一致性要求：强一致
原因：同一设备同一时段不能出现两条有效预约
推荐处理方式：后端校验 + 数据库约束 + 事务/锁

取消预约
一致性要求：强一致
原因：取消后设备时段应立即释放，避免状态混乱
推荐处理方式：状态更新放入事务，记录操作日志

发送通知
一致性要求：最终一致
原因：通知晚到可接受，不应阻塞预约主流程
推荐处理方式：异步任务/消息队列 + 失败重试

统计报表
一致性要求：最终一致
原因：报表允许分钟级或小时级延迟
推荐处理方式：定时汇总或事件驱动更新

权限判断
一致性要求：强一致
原因：越权访问会造成安全问题
推荐处理方式：后端统一鉴权与授权校验

搜索索引更新
一致性要求：最终一致
原因：搜索结果短暂延迟通常可接受
推荐处理方式：异步索引更新，必要时回源查询

【考试提示】

判断同步还是异步时，不要只看实现是否方便，要看业务结果是否允许延迟修正。预约占用和权限判断通常不能延迟修正，通知和统计通常可以。

6.9 技术债：有意接受的风险要被记录

技术债不是“代码写得差”的同义词。更准确地说，技术债是团队为了短期目标有意接受的工程风险。它可能是合理的，但必须被记录和管理。

例如，首版实验室预约系统为了赶上展示，可能暂时不接入真实短信服务，只实现邮件通知和站内消息。这不一定错误，但它应该被记录为技术债：当前为什么接受，影响范围是什么，未来什么时候补齐。

6.9.1 技术债记录表示例

字段	示例
技术债名称	首版未接入短信通知，仅支持邮件和站内消息
为什么接受	项目周期有限，短信接口申请和费用审批耗时较长；首版核心目标是验证预约与审批闭环
影响范围	用户可能不能及时收到移动端提醒；紧急设备变更通知触达不够稳定
风险	邮件被忽略或延迟查看，可能导致学生错过实验安排
何时偿还	系统进入试运行，且通知失败率或迟到率超过设定阈值时，补接短信或企业微信通知

观察指标	通知失败率、通知平均送达时间、用户未读通知比例、因通知迟到产生的投诉次数

6.9.2 技术债管理原则

技术债可以接受，但应满足三个条件：

1. 债务是有意接受的，不是无意识遗漏。
2. 债务有记录，说明影响范围和风险。
3. 债务有触发条件，知道什么时候必须处理。

没有记录的技术债会变成系统隐患；有记录、有边界、有偿还条件的技术债，才是可管理的工程取舍。

【项目提示】

技术债不是“写得不好”的委婉说法，而是“为了当前目标有意接受的风险”。如果没有原因、影响范围和偿还条件，就不能算可管理的技术债。

6.10 项目实践：为你的系统写 3 条 ADR

本章学习后，课程项目至少应补充 3 条 ADR。下面给出可直接使用的填写提示。

ADR-001：首版架构风格选择

建议回答：

- 当前系统首版采用什么架构风格？
- 为什么该风格适合当前团队、周期和业务边界？
- 为什么暂不采用微服务或其他更复杂方案？
- 该方案的主要风险是什么？
- 后续根据什么指标判断是否需要调整？

可选标题：

ADR-001：首版采用模块化单体架构

ADR-002：关键一致性策略

建议回答：

- 系统中哪条业务链路必须强一致？
- 为什么不能最终一致？
- 使用什么机制保障一致性？
- 该机制带来什么性能或实现成本？
- 如何测试并发情况下的一致性？

可选标题：

ADR-002：预约占用采用后端强一致控制

ADR-003：外部依赖或权限模型处理

建议回答：

- 系统依赖哪些外部服务，例如统一认证、邮件、短信、文件存储？
- 外部服务失败时主流程如何处理？
- 权限校验放在前端、后端还是统一网关？
- 为什么这样设计？
- 如何记录失败、重试和审计？

可选标题：

ADR-003：通知服务异步处理并支持失败重试

ADR-003：权限校验统一在后端完成

6.10.1 ADR 填写模板

可直接复制到项目文档中使用：

字段	填写内容
ADR 编号与标题	ADR-00X：……
背景	当前业务场景、团队条件、时间限制、部署环境、质量要求
决策	最终选择的架构方案或结构策略
放弃方案	至少写出一个没有采用的方案，并说明原因
理由	当前方案为什么更匹配约束
风险	当前方案可能带来的代价或隐患
观察指标	后续根据哪些现象或数据判断是否需要调整

6.10.2 技术债记录模板

字段	填写内容
技术债名称	……
为什么接受	为了交付、验证、成本、资源限制等原因暂时接受
影响范围	影响哪些模块、用户、质量属性或后续开发
风险	如果长期不处理会发生什么
何时偿还	触发偿还的时间、版本、指标或业务条件
观察指标	失败率、耗时、缺陷数、投诉数、修改影响范围等

6.10.3 架构决策自查清单

提交架构方案前，可逐项检查：

- 是否说明了核心质量属性，而不只是功能清单？
- 是否区分了技术栈和架构理由？
- 是否写清了模块边界和依赖方向？
- 是否说明哪些链路必须强一致，哪些可以最终一致？
- 是否解释了为什么采用当前架构风格？
- 是否写出了放弃方案？
- 是否记录了已知风险和观察指标？
- 是否识别了至少一项技术债？

【项目提示】第 6 章内容较多，复习时不需要把所有架构风格都背成名词表。优先掌握三条主线：质量属性如何驱动架构决策；为什么小团队首版通常优先模块化单体；怎样用 ADR 记录选择、放弃方案和风险。

本章与后续学习的关系

本章是架构设计主干。下一章将从测试和质量保障角度验证这些架构决策是否成立。

6.11 本章小结

1. 架构设计的核心不是选择流行技术，而是在约束下做关键结构决策。
2. 技术栈只说明用了什么工具，架构方案必须说明边界、协作、质量属性和取舍理由。
3. 质量属性会直接驱动结构选择：预约占用要求强一致，通知失败适合异步处理，权限安全要求后端统一控制。
4. 模块化单体通常更适合小团队首版项目；微服务需要清晰业务边界和成熟工程能力，否则容易变成分布式单体。
5. ADR 和技术债记录能让架构选择可审查、可追踪、可演进。

6.12 复习题与参考答案要点

6.12.1 基础概念题（3 题）

1. 什么是软件架构？它与技术栈清单有什么区别？

参考答案要点：

- 软件架构是系统中关键结构决策的集合。
- 包括主要组成单元、协作方式、依赖关系、数据流向和质量属性保障方式。
- 技术栈清单只说明使用了哪些工具或框架。
- 架构方案必须解释为什么这样组织系统，以及这种组织如何满足质量属性。

2. 什么是质量属性？请用预约系统举例说明。

参考答案要点：

- 质量属性描述系统以什么质量完成工作。
- 功能需求说明做什么，质量属性说明怎样做得可靠、安全、可维护。
- 预约提交是功能需求。
- 同一设备同一时段不能产生两个有效预约是一致性要求。
- 通知服务失败不影响预约主流程是可用性要求。

3. 什么是 ADR？为什么 ADR 要记录放弃方案？

参考答案要点：

- ADR 是架构决策记录，用于记录重要架构选择及其理由。
- 应包含背景、决策、放弃方案、理由、风险和观察指标。
- 放弃方案体现比较和权衡，说明不是随意选择。
- 后续条件变化时，可根据放弃方案判断是否重新评估。

6.12.2 简答题（2 题）

1. 为什么小团队首版项目通常更适合模块化单体，而不是直接微服务？

参考答案要点：

- 小团队人少，服务拆分会增加沟通和接口协调成本。
- 首版业务边界尚未稳定，过早拆分容易拆错。
- 微服务需要独立部署、监控、自动化测试、链路追踪等工程能力。
- 模块化单体部署和调试简单，同时可通过内部模块边界保留演进空间。

2. 为什么“拆成多个服务”不等于微服务？

参考答案要点：

- 微服务强调业务边界清晰、数据归属明确、服务可独立部署和独立演进。
- 如果多个服务共享数据库、必须同时发布、接口强耦合，就是分布式单体。
- 分布式单体既失去单体简单性，又承担分布式复杂性。
- 判断关键不在服务数量，而在自治性、边界和工程治理能力。

6.12.3 案例分析题（2 题）

1. 预约成功后邮件服务超时，系统是否应该回滚预约？请给出架构判断。

参考答案要点：

- 不应直接回滚预约，除非通知本身被定义为预约成立的必要条件。
- 预约占用是核心业务结果，应保持强一致。
- 邮件通知属于外部依赖，可异步处理。
- 合理方案是记录通知任务，后台发送，失败后重试并记录原因。

- 风险是用户可能延迟收到通知，可通过站内消息、失败告警和重试机制降低风险。

2. 某小组把预约、审批、通知、统计拆成四个服务，但四个服务共享同一个数据库，每次发布都必须一起上线。请判断这是不是微服务，并说明原因。

参考答案要点：

- 这不是成熟微服务，更接近分布式单体。
- 服务共享数据库，说明数据边界不清。
- 必须一起发布，说明服务不能独立演进。
- 多服务增加了网络调用和部署复杂度，但没有获得服务自治收益。
- 更合理的首版方案可能是模块化单体，内部保持清晰边界。

6.12.4 项目实践题（1 题）

1. 为本组课程项目写一条 ADR，主题自选。

参考答案要点：

- 标题明确，例如“首版采用模块化单体”或“通知采用异步处理”。
- 背景应说明团队、周期、业务场景和约束。
- 决策应写清最终选择。
- 放弃方案应至少列出一个，并说明原因。
- 理由应连接背景和决策。
- 风险应承认当前方案可能的问题。
- 观察指标应说明后续根据什么判断是否调整。

第 7 章 软件测试、质量保障与安全验证

7.1 本章导读：测试不是最后找 Bug，而是验证架构判断

软件测试常被误认为开发完成后的检查工作。实际上，能不能测试，往往在需求和架构阶段已经决定。若系统没有清晰边界、状态不可观察、依赖无法替换、业务规则散落在页面和数据库脚本中，测试会变得困难且低效。

实验室预约系统中的关键质量问题包括：预约冲突是否被阻止，权限越权是否被拒绝，通知失败是否不影响主流程，审批状态是否可追踪，数据统计是否可信。这些问题不能只靠人工点页面发现，而应通过单元测试、集成测试、接口测试、安全测试和质量门禁形成保障。

本章重点说明测试层级、可测试性、质量门禁、安全验证和缺陷管理。目标不是掌握所有测试工具，而是理解质量如何进入软件工程全过程。

【你要记住】测试不是系统完成后的补救动作，而是从需求、架构到交付持续验证工程判断的机制。

7.2 一个反面案例：所有逻辑都在 Controller 里

某预约系统把所有逻辑写在一个 Controller 中：读取请求参数、判断权限、查询设备、校验冲突、写数据库、发送通知、返回页面结果。演示阶段看起来能运行，但一写测试就暴露问题。

首先，业务规则无法单独测试。要测试预约冲突，必须启动 Web 服务、连接数据库、准备用户会话。其次，外部依赖无法替换。邮件服务不可用时，测试会随机失败。再次，异常路径难以覆盖。权限不足、设备维护、并发提交、通知失败都混在一个方法里。最后，缺陷定位困难。测试失败时，很难判断是参数解析、业务规则、数据库事务还是邮件调用出了问题。

改进方案是把业务规则放入服务层，把数据库访问封装到 Repository，把外部通知封装为适配器，把权限校验作为统一机制，把关键状态写入可观察日志。这样测试才能分层展开。

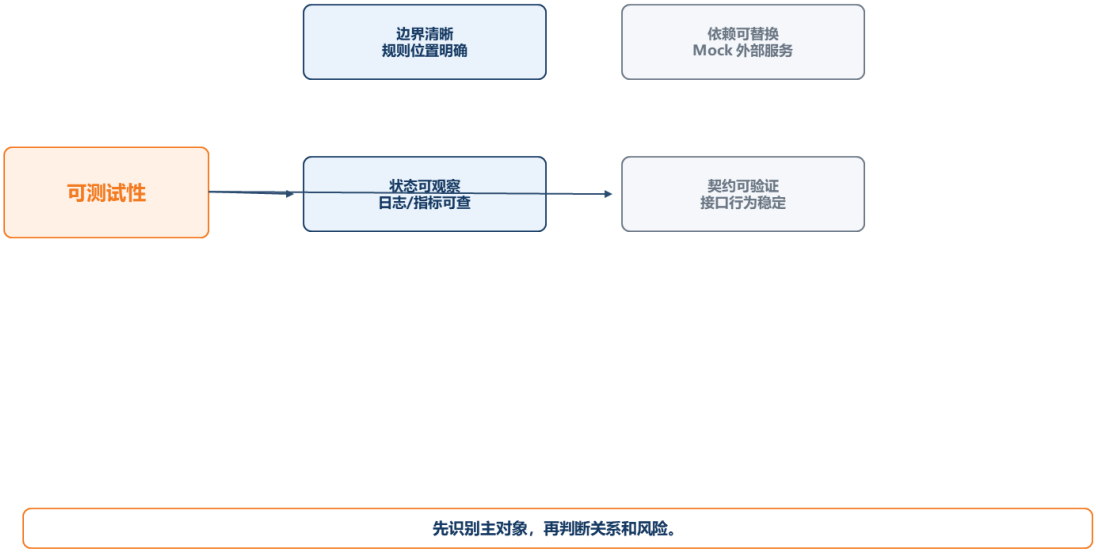
7.3 可测试性：架构给测试创造条件

可测试性是系统容易被验证的程度。它不是测试人员单独决定的，而是由架构边界、依赖管理、状态可观察性和接口契约共同决定。

对实验室预约系统来说，若“冲突检测”是一个独立业务服务，就可以构造不同时间段、不同设备、不同预约状态进行单元测试；若通知发送通过接口抽象，就可以在测试中替换为 Mock；若系统记录状态转换日志，就能在失败时判断流程停在哪一步；若接口有明确契约，就能验证前后端和第三方服务的一致性。

【易错点】测试困难通常不是因为测试框架不够强，而是因为代码边界和依赖关系没有设计好。

图 7-1 架构可测试性条件



图注：本图说明测试策略要回到架构结构，而不是只讨论测试工具。

Mock、Stub 与契约测试

Mock 和 Stub 都用于在测试中替代真实依赖。Stub 主要返回预设结果，例如让“通知服务”固定返回成功或失败；Mock 还会验证调用是否发生，例如检查系统是否在预约通过后调用了通知接口。契约测试关注服务之间的接口约定是否一致，适合前后端、后端与第三方服务之间的协作。

在实验室预约系统中，通知服务、统一认证、文件存储都可以在测试中用替身处理。这样做不是为了逃避真实集成，而是为了先验证业务规则，再在集成测试中验证真实依赖。

【易错点】Mock 不是把测试做假，而是让测试聚焦当前对象。真正上线前仍然需要集成测试和端到端测试。

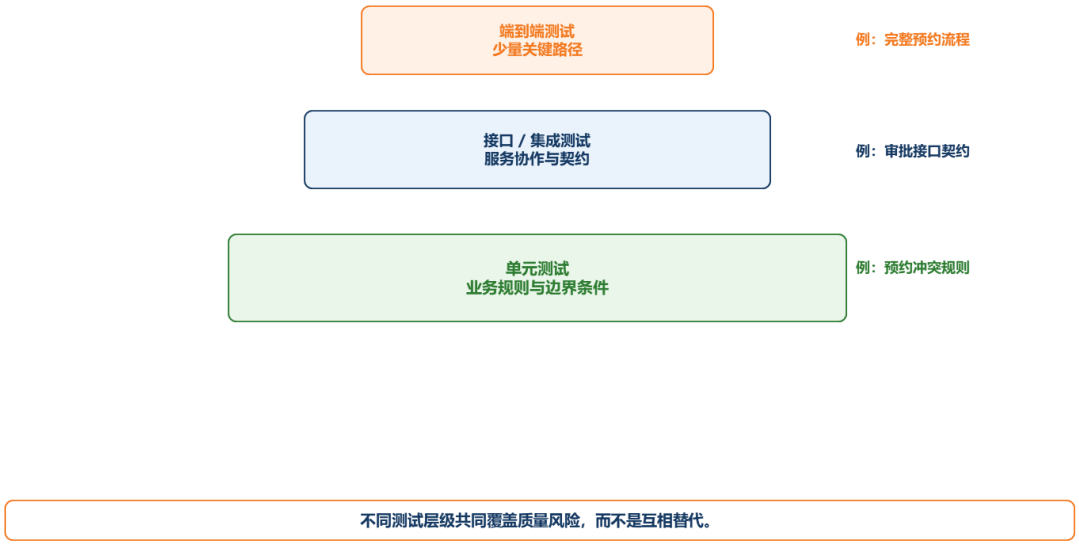
7.4 测试金字塔：不同层级验证不同问题

测试金字塔通常包括单元测试、集成测试、接口/服务测试、端到端测试。底层测试数量多、运行快，适合验证业务规则和函数逻辑；上层测试数量少、成本高，适合验证关键流程和系统协作。

在实验室预约系统中，冲突检测规则适合单元测试；预约服务与数据库事务适合集成测试；预约接口和权限返回适合接口测试；“学生提交预约—教师审批—学生查看结果”适合端到端测试。

若所有测试都依赖浏览器端到端点击，测试成本会很高且定位困难。若只有单元测试，又无法保证模块之间协作正确。合理策略是分层验证。

图 7-2 测试金字塔与预约系统示例



图注：本图说明不同测试层级不是互相替代，而是共同覆盖不同质量风险。

7.5 从质量属性到测试场景

质量属性要变成测试，必须写成可验证场景。

质量属性	场景	测试方式
一致性	两个用户同时预约同一设备同一时段	并发接口测试、数据库约束检查
安全性	学生访问审批接口	权限测试，期望返回 403
可用性	通知服务不可用	Mock 通知失败，验证预约仍保存
性能	查询一周可预约时间段	接口性能测试
可维护性	新增审批规则	单元测试覆盖规则服务

这个表说明，质量属性不是写在文档里的形容词，而要进入测试设计。

三类关键测试用例样例

预约冲突并发测试

前置条件：设备 A 在周三上午可预约

操作：两个用户同时提交同一时段预约

期望结果：只有一个请求成功，另一个返回冲突；数据库只有一条有效预约

权限越权测试
前置条件：学生账号已登录
操作：直接请求审批接口
期望结果：返回 403；不产生审批记录；记录权限拒绝日志

通知失败测试
前置条件：通知服务模拟超时
操作：学生提交预约并通过校验
期望结果：预约记录保存成功；通知任务标记失败或待重试；主流程不回滚

这些测试用例体现了第 6 章的架构判断：一致性要求、权限边界和异步通知都必须能被验证。

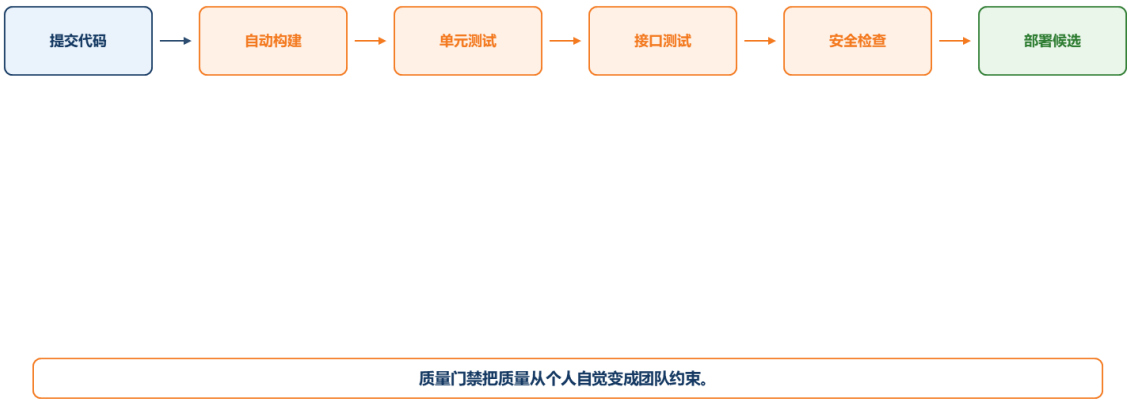
7.6 质量门禁：把质量要求放进开发流程

质量门禁是团队在代码合并、构建、发布前设置的质量检查条件。常见门禁包括代码评审、静态检查、单元测试通过、接口测试通过、安全扫描、测试覆盖率阈值和构建成功。

课程项目可以采用轻量门禁：主分支合并前至少一名成员评审；关键业务规则必须有单元测试；接口变更必须更新说明；部署前必须跑通预约闭环；权限接口必须做越权测试。

门禁的意义不是增加形式，而是防止低质量代码在团队中扩散。没有门禁的项目，常见情况是“每个人本地都能跑，合到一起就坏”。

图 7-3 CI/CD 中的质量门禁



图注：本图把质量门禁嵌入 CI/CD 流程，说明质量要求应在交付链路中自动执行。

质量门禁清单

阶段	门禁内容	不通过时的处理

合并前	代码评审、关键单元测试、接口说明更新	不允许合并主分支
发布前	预约闭环、权限测试、通知失败测试、数据库脚本检查	暂缓发布，修复后重新验证
答辩前	从零部署、测试账号、演示数据、已知问题说明	补齐交付材料，避免现场不可复现

【项目提示】课程项目的质量门禁可以很轻量，但必须写清“检查什么、谁负责、不通过怎么办”。

7.7 缺陷管理：Bug 不是一句“有问题”

一个可处理的缺陷报告应包含：标题、环境、复现步骤、实际结果、期望结果、严重程度、影响范围、截图或日志、负责人和状态。

例如，“预约失败”不是好缺陷描述。更好的写法是：“在 Chrome 浏览器中，学生 A 预约设备 X 的周三上午时段，系统提示成功，但教师端看不到待审批记录。复现步骤为……实际结果为……期望结果为……日志显示预约记录状态为草稿。”这样的缺陷才能被定位和修复。

缺陷还应分类：需求缺陷、设计缺陷、编码缺陷、数据缺陷、环境缺陷、权限缺陷。分类有助于发现系统性问题。

7.8 安全验证：权限、输入和数据边界

软件安全不是只在上线前扫描一次。课程项目至少应关注三类安全验证：认证授权、输入校验和敏感数据保护。

认证授权要验证未登录、角色不匹配、越权访问和接口绕过。输入校验要关注参数篡改、SQL 注入风险、文件上传限制和异常字符。敏感数据保护要关注密码、联系方式、身份信息和日志脱敏。

在实验室预约系统中，学生不能访问审批接口，实验员不能修改系统管理员权限，普通用户不能通过修改预约 ID 查看他人记录。所有这些都应进入测试用例。

安全验证还应覆盖输入边界和日志审计。例如，预约 ID 不能通过修改 URL 查看他人记录；上传附件要限制类型和大小；日志中不应打印明文密码、身份证号或访问令牌；审批接口需要记录操作者、操作时间和结果。安全测试的核心不是“找黑客工具”，而是验证系统是否守住角色、接口和数据边界。

【考试提示】安全测试题常要求结合角色、接口、数据和日志说明。回答时不要只写“加强安全”，要写具体验证点。

7.9 AI 辅助测试：可以提效，但不能替代判断

AI 工具可以帮助生成测试用例、构造边界输入、解释日志和补充异常场景。但 AI 生成的测试必须由人审查，尤其是业务规则、安全边界和数据一致性问题。

例如，可以提示 AI 为“提交预约申请”生成正常场景、冲突场景、权限不足、设备维护、通知失败、并发提交等测试用例。项目组再根据真实规则筛选、补充和执行。

AI 辅助测试的价值在于扩大思考范围，风险在于生成看似完整但不符合业务规则的用例。因此它应作为辅助，而不是质量责任的替代者。

7.10 项目实践：为预约系统设计测试包

课程项目建议提交一个轻量测试包。

内容	要求
测试范围	核心用例、关键质量属性、安全边界
单元测试	至少覆盖 2 个核心业务规则
接口测试	覆盖预约、审批、权限、通知
端到端测试	覆盖一个完整业务闭环
缺陷清单	至少记录并分析 3 个缺陷
质量门禁	说明合并和发布前检查条件

测试包的重点不是数量，而是能说明系统质量如何被验证。

本章与后续学习的关系

本章说明质量如何被验证。下一章将继续讨论系统如何发布、回滚、观察和复盘。

7.11 本章小结

1. 可测试性由架构边界、依赖替换、状态观察和接口契约共同决定。
2. 测试金字塔用于把不同风险分配到不同测试层级。
3. 质量属性必须转化为测试场景和度量标准。
4. 质量门禁把质量控制嵌入开发与发布流程。
5. 安全验证和 AI 辅助测试都必须回到具体规则和责任边界。

7.12 复习题与参考答案要点

【题型：名词解释｜难度：基础】基础概念题 1：什么是可测试性?

参考答案要点：系统容易被验证的程度，受边界清晰、依赖可替换、状态可观察、契约可验证等因素影响。

【题型：名词解释｜难度：基础】基础概念题 2：测试金字塔包括哪些层级?

参考答案要点：单元测试、集成测试、接口/服务测试、端到端测试，底层多且快，上层少且成本高。

【题型：名词解释｜难度：基础】基础概念题 3：什么是质量门禁?

参考答案要点：在代码合并、构建、发布前设置的质量检查条件，如测试通过、评审、安全扫描和构建成功。

【题型：简答题｜难度：中等】简答题 1：为什么测试困难往往是架构问题?

参考答案要点：若边界混乱、依赖不可替换、状态不可观察、业务规则散落，测试就难以隔离、构造和定位。

【题型：简答题｜难度：中等】简答题 2：如何把“系统要安全”转化为可测试需求?

参考答案要点：定义角色权限、接口访问规则、未授权返回、日志记录、输入校验和越权测试用例。

【题型：案例分析｜难度：拔高】案例分析题：预约系统通知服务不可用导致预约接口失败。请分析设计和测试问题。

参考答案要点：通知与主流程耦合过强；应考虑异步通知或失败重试；测试应覆盖通知失败场景，验证预约记录是否正确保存。

【题型：项目应用｜难度：中等】项目实践题：为课程项目设计一条质量门禁。

参考答案要点：说明触发位置、检查内容、通过标准、失败处理和责任人。

第 8 章 发布、交付、运维与可观测性

8.1 本章导读：完成开发不等于完成交付

软件系统真正进入使用，不是在代码写完时，而是在它能够稳定部署、可回滚、可监控、可排障、可持续演进时。很多学生项目在本地运行良好，但一到演示或部署环境就出现数据库连接失败、配置丢失、接口跨域、文件路径错误、版本不一致等问题。这些都属于交付工程的一部分。

实验室预约与设备管理系统如果要给学院试用，需要面对账号配置、数据库初始化、日志查看、邮件服务、权限配置、备份恢复、版本升级和故障处理。若没有发布与运维设计，系统只能停留在“开发机能跑”。

本章介绍持续集成、持续交付、发布策略、回滚、配置管理、可观测性和故障复盘。目标是让学生理解：交付能力也是软件体系结构和工程能力的重要组成部分。

【你要记住】交付不是把代码拷到服务器，而是让系统以可控方式进入真实运行环境。

8.2 一个反面案例：演示前一晚的手工发布

某项目组在答辩前一晚准备部署预约系统。后端在成员 A 电脑上能运行，前端在成员 B 电脑上能运行，数据库结构在群里传了一个 SQL 文件。部署到服务器后，后端无法连接数据库，因为配置文件路径不同；前端接口地址仍指向本地；邮件密码写在代码中，被提交到仓库；数据库缺少测试数据；系统报错后没有日志，只能猜测问题。

最后项目组花了数小时手工修改配置、重新导入数据库、临时关闭通知功能。系统勉强演示成功，但没有人能保证下次还能部署出来。

这类问题说明，发布交付不是最后一步小事，而是需要在开发过程中持续建设的能力。部署脚本、环境配置、版本记录、日志和回滚方案都应尽早准备。

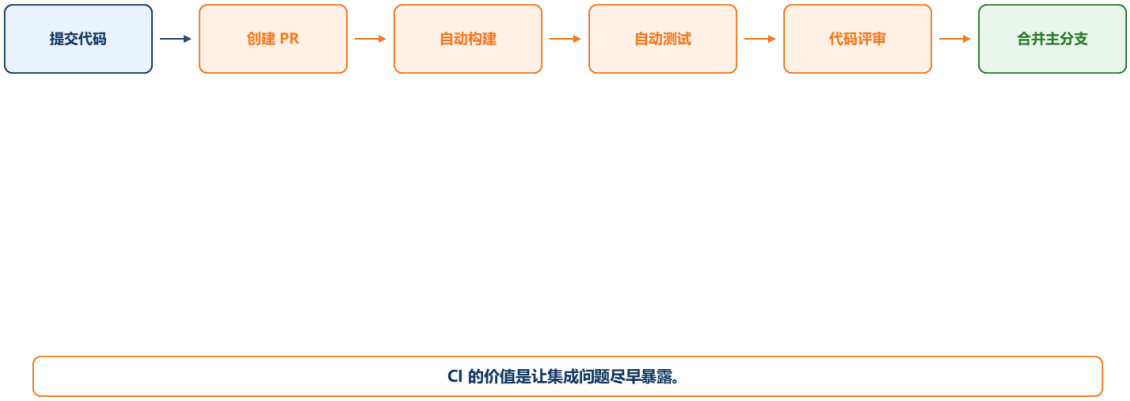
8.3 持续集成：让问题尽早在公共环境暴露

持续集成（CI）强调团队成员频繁合并代码，并通过自动化构建和测试尽早发现集成问题。它解决的不是“个人代码能不能运行”，而是“团队代码合在一起是否仍然可用”。

对课程项目而言，CI 可以简化为：每次合并到主分支前自动检查代码能否构建、关键测试是否通过、依赖是否完整。即使没有复杂平台，也应保持固定集成节奏，不要等到最后一周才合并。

实验室预约系统的 CI 检查可以包括：后端测试通过、前端构建通过、接口契约未破坏、数据库迁移脚本可执行、关键配置项完整。

图 8-1 持续集成基本流程



图注：本图说明持续集成如何把问题前移，避免把冲突积累到发布前。

8.4 持续交付：让版本随时具备发布候选状态

持续交付（CD）强调系统在通过必要检查后，能够随时形成可发布版本。它不一定意味着每次提交都自动上线，但意味着构建、测试、打包、配置和部署过程可重复。

课程项目可以把持续交付理解为：任何一个正式版本都能从仓库、脚本和说明文档重新部署出来。版本包应包含前后端构建产物、数据库初始化脚本、配置说明、启动命令、测试账号和回滚说明。

若系统只能依赖某个成员电脑上的隐含配置运行，就不具备交付能力。

【项目提示】答辩前至少做一次“从零部署演练”：换一台机器或新目录，按照部署说明重新运行系统。

需要区分持续交付和持续部署。持续交付强调版本随时具备发布候选状态，但是否上线可以由人工决策；持续部署则是在通过门禁后自动上线。课程项目通常不需要自动上线，但需要做到构建、配置、部署步骤可重复。

从零部署检查表

检查项	要求
运行环境	明确 JDK/Node/数据库版本和端口
配置文件	提供示例配置，敏感信息不入库
数据库	提供初始化脚本和测试数据
启动步骤	前端、后端、数据库启动顺序清楚
测试账号	至少包含学生、审批人、管理员
日志位置	说明错误日志和业务日志在哪里查看
回滚方式	说明如何恢复上一个可运行版本

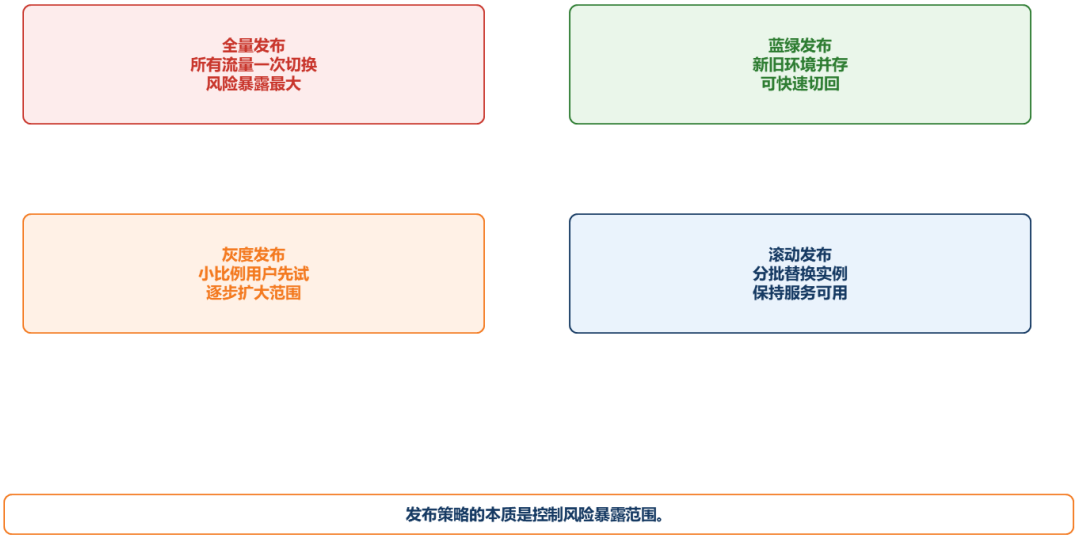
8.5 发布策略：不要把所有用户一次性暴露给新版本

常见发布策略包括全量发布、蓝绿发布、灰度发布、滚动发布和金丝雀发布。课程项目不一定需要完整实现这些策略，但应理解它们解决的问题：降低发布风险、控制影响范围、支持快速回滚。

全量发布简单，但一旦新版本有问题，所有用户都会受影响。蓝绿发布准备两套环境，新版本在备用环境验证后切换流量。灰度或金丝雀发布先让少量用户使用新版本，观察指标稳定后再扩大范围。滚动发布逐步替换实例，适合多实例服务。

对实验室预约系统而言，若只是课程演示，可以使用简单版本发布；若给学院试用，至少要有备份、回滚和发布记录。若修改预约冲突规则，应先在测试环境验证，再安排低风险时间窗口发布。

图 8-2 常见发布策略对比



图注：本图对比全量、蓝绿、灰度和滚动发布，帮助判断不同策略如何控制上线风险。

8.6 回滚与数据库变更：最容易被低估的交付风险

代码回滚相对容易，数据库回滚往往更困难。若新版本增加字段、改变状态含义、迁移历史数据，回滚时可能遇到数据不兼容。

实验室预约系统若把预约状态从“0/1/2”改成“待审批/已通过/已拒绝/已取消”，必须考虑旧数据如何迁移、接口如何兼容、回滚后新状态是否丢失。若只修改代码不管理数据库版本，系统很容易在升级后出现不可恢复错误。

因此，发布计划应包括数据库迁移脚本、备份策略、回滚条件和回滚步骤。对课程项目而言，至少要在部署说明中写清数据库初始化和升级方法。

【易错点】“能上线”不代表“能安全上线”。没有回滚方案的发布，本质上是在赌新版本不会出错。

数据库迁移示例：预约状态字段调整

步骤	说明
变更前	状态字段使用 0/1/2，含义依赖代码注释

变更目标	改为 PENDING/APPROVED/REJECTED/CANCELLED
迁移动作	新增枚举映射脚本，先备份旧表，再执行数据转换
兼容风险	旧接口若仍返回数字状态，前端可能显示错误
回滚方案	保留备份表和旧字段映射，确认新版本稳定后再清理

【考试提示】发布与交付题常考“为什么数据库回滚难”。答题时要提到数据已经变化、历史记录可能不兼容、回滚必须配合备份和迁移脚本。

8.7 配置管理与环境一致性

配置包括数据库地址、账号密码、邮件服务、文件目录、端口、第三方接口、日志级别等。配置不应散落在代码中，更不应把真实密码提交到仓库。

环境通常包括开发环境、测试环境和生产/演示环境。不同环境配置可以不同，但结构应一致。例如都使用相同的配置项名称、相同的数据库迁移脚本、相同的启动方式。

实验室预约系统应把数据库连接、邮件服务、统一认证地址、文件上传目录放入配置文件或环境变量，并提供示例配置。敏感信息应使用占位符或本地私密配置，不应公开。

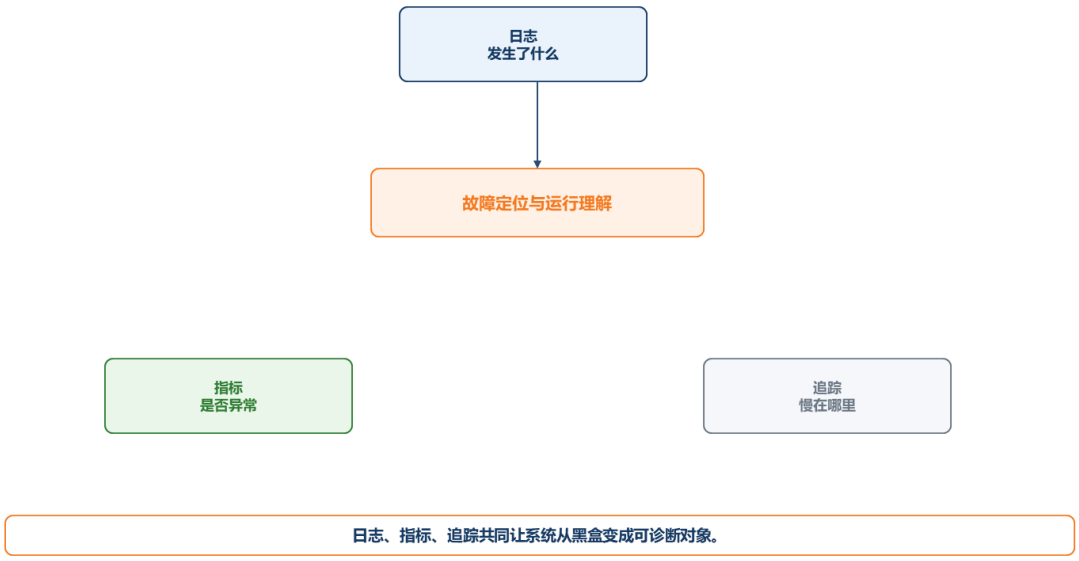
8.8 可观测性：出问题时能看见系统内部发生了什么

可观测性通常包括日志、指标和链路追踪。日志记录事件和上下文，指标反映系统状态和趋势，链路追踪帮助理解一次请求经过哪些服务。

课程项目可以从日志和关键指标开始。实验室预约系统至少应记录：登录用户、预约提交、审批操作、权限拒绝、通知失败、异常堆栈和关键状态变化。指标可以包括预约数量、审批通过率、通知失败次数、接口错误率和响应时间。

可观测性的目标不是收集大量数据，而是在故障发生时能回答：发生了什么，影响谁，在哪个环节，是否仍在扩大，如何恢复。

图 8-3 可观测性三支柱



图注：本图说明日志、指标、追踪共同让系统从黑盒变成可诊断对象。

预约系统日志与指标示例

类型	示例	用途
业务日志	reservation.submit.success，记录用户、设备、时间段、预约 ID	追踪预约是否成功
权限日志	auth.denied.count，记录角色、接口、时间	发现越权访问和配置错误
通知指标	notification.failed.count	判断通知服务是否异常
性能指标	api.reservation.query.duration	观察预约查询是否变慢
错误日志	异常栈、请求 ID、用户角色	支持故障定位

可观测性要与第 7 章测试形成闭环：测试在上线前暴露问题，日志和指标在运行中暴露问题。

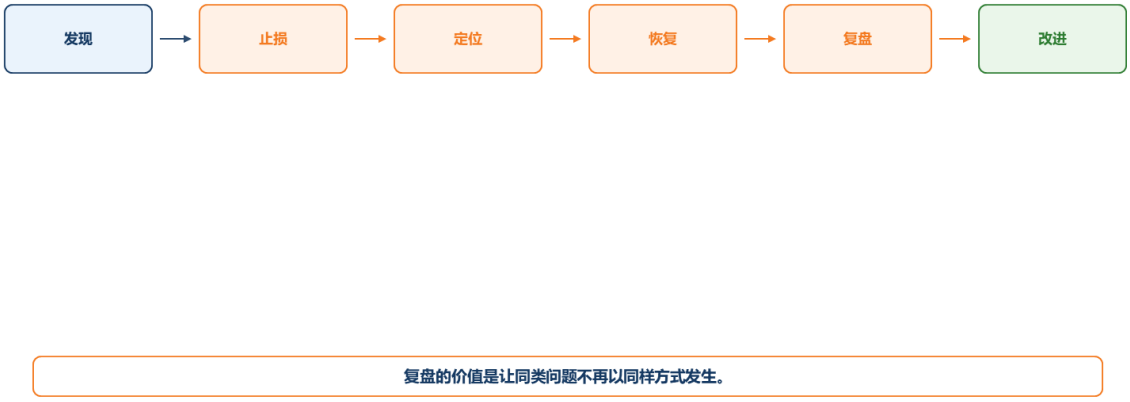
8.9 故障处理与复盘：把事故变成改进

系统故障不可完全避免。重要的是发现、止损、定位、恢复和复盘。

一个基本故障处理流程包括：确认现象、评估影响、暂停危险操作、保留日志、恢复服务、通知相关人员、记录根因、制定改进。复盘不应只是追究谁写错代码，而应分析为什么缺陷没有被测试、评审或监控提前发现。

例如，预约冲突故障发生后，复盘可能发现：前端做了冲突提示，但后端没有事务保护；测试没有并发场景；日志没有记录冲突原因；发布前没有回归测试。改进措施应对应这些根因。

图 8-4 故障复盘闭环



图注：本图说明故障处理不是到恢复为止，还要通过复盘把经验转化为质量门禁和监控改进。

8.10 项目实践：提交一份交付与运维说明

课程项目建议提交一份轻量交付说明。

内容	要求
环境说明	运行环境、依赖版本、端口
部署步骤	前端、后端、数据库、配置
初始化数据	测试账号、基础设备数据
发布记录	版本号、变更内容、负责人
回滚方案	何时回滚、如何恢复
日志与监控	日志位置、关键指标、故障排查
已知限制	暂未解决问题和后续计划

这份说明能让系统从“某个成员电脑上能跑”变成“别人也能部署和维护”。

本章与后续学习的关系

本章完成从开发到运行的闭环。课程项目最终应把需求、建模、架构、测试和交付材料整合为可答辩成果。

8.11 本章小结

1. 交付能力包括构建、部署、配置、版本、回滚和运行诊断。
2. 持续集成让团队尽早发现集成问题，持续交付让版本保持可发布状态。
3. 发布策略的核心是控制风险范围，回滚方案是安全发布的必要条件。
4. 可观测性帮助系统在故障时可定位、可恢复、可改进。

5. 课程项目应至少提供可重复部署说明、基础日志和版本记录。

8.12 复习题与参考答案要点

【题型：名词解释｜难度：基础】基础概念题 1：什么是持续集成？

参考答案要点：团队频繁合并代码，并通过自动化构建和测试尽早发现集成问题。

【题型：名词解释｜难度：基础】基础概念题 2：什么是可观测性？

参考答案要点：通过日志、指标和追踪理解系统运行状态、定位故障和支持恢复的能力。

【题型：名词解释｜难度：基础】基础概念题 3：蓝绿发布和灰度发布的区别是什么？

参考答案要点：蓝绿发布在两套环境间切换流量；灰度发布逐步扩大用户范围，观察稳定后再全量。

【题型：简答题｜难度：中等】简答题 1：为什么数据库变更比代码回滚更难？

参考答案要点：数据结构和历史数据可能已改变，回滚可能造成数据丢失或不兼容，需要迁移、备份和回滚计划。

【题型：简答题｜难度：中等】简答题 2：课程项目为什么也需要部署说明？

参考答案要点：保证系统可重复运行，减少环境依赖，支持答辩演示、评审复现和后续维护。

【题型：案例分析｜难度：拔高】案例分析题：某系统上线后通知失败但没有日志，无法判断预约是否成功。请提出改进。

参考答案要点：记录预约状态变化、通知调用结果和异常日志；通知失败不应破坏主流程；增加指标和告警；设计重试或补偿机制。

【题型：项目应用｜难度：中等】项目实践题：为课程项目写一份交付清单。

参考答案要点：包含运行环境、部署步骤、数据库初始化、配置说明、版本记录、回滚方案和日志位置。

附录 A 核心术语速查表

术语	简要说明
FR	功能需求，回答系统要做什么。
QA	质量属性，回答系统要做到什么程度。
C	约束条件，回答系统必须在什么边界内完成。
软件复杂性	需求变化、团队协作、技术组合和运行交付约束共同造成的工程难度。
架构时间账	当前架构选择未来可能付出的维护、返工和演进成本。
过程模型	组织软件开发节奏、反馈和风险暴露方式的方法。
瀑布模型	按阶段顺序推进的开发方式，适合需求稳定、验证边界清楚的项目。
原型模型	通过早期原型澄清需求和交互，适合用户表达不清的场景。
增量开发	每次交付增加一部分可用功能，强调可交付范围逐步扩大。
迭代开发	基于反馈反复改进同一目标，强调质量和理解逐步深化。
螺旋模型	围绕风险分析循环推进的过程模型，适合技术和业务风险较高的项目。
Scrum	常见敏捷框架，以 Sprint、Backlog、站会和评审组织短周期交付。
WBS	工作分解结构，把项目目标拆成可估算、可分配、可验收的工作包。
里程碑	项目中的关键可验证状态，不应只是日期标记。
风险登记表	记录风险、触发信号、影响、应对动作和责任人的项目管理工具。
Ticket	团队跟踪单个需求、缺陷或任务的轻量工作项。
PR	Pull Request，代码合并前用于评审、讨论和自动检查的协作机制。
利益相关者	会影响系统或被系统影响的人、组织或外部系统。
典型用户	用于代表一类目标用户特征、目标和使用场景的分析对象。
用例	从用户目标出发描述系统交互过程和业务规则的需求表达方式。
SRS	软件需求规格说明书，用于记录可开发、可测试、可评审的需求约定。

原型	用于验证需求、交互和流程理解的低成本可视化或可运行模型。
C4 模型	从上下文、容器、组件和代码层次表达系统结构的方法。
UML	统一建模语言，用于表达用例、类、状态、活动、时序等系统视角。
ERD	实体关系图，用于表达数据实体、属性和关系。
DFD	数据流图，用于表达数据如何在处理过程、外部实体和存储之间流动。
ADR	架构决策记录，用于记录背景、决策、放弃方案、理由、风险和观察指标。
模块化单体	单体部署但内部模块边界清晰的架构组织方式。
六边形架构	通过端口和适配器隔离业务核心与外部技术依赖的架构思想。
DDD 限界上下文	在明确业务语言边界内组织模型和职责的领域驱动设计概念。
SOA	面向服务架构，强调通过服务组织共享能力。
微服务	服务边界清晰、数据归属明确、可独立部署和演进的分布式架构方式。
分布式单体	拆成多个服务但仍高度耦合、共享数据、不能独立演进的错误形态。
强一致	业务结果必须立即一致，通常用于预约占用、支付、权限等关键链路。
最终一致	允许短时间不一致，但通过补偿、重试或异步处理最终达到一致。
技术债	当前为速度或约束接受的技术风险，未来需要偿还。
测试金字塔	按单元、接口、集成、端到端等层级组织测试投入的经验模型。
质量门禁	在构建、测试、发布过程中设置的质量检查条件。
契约测试	验证服务消费者和提供者之间接口约定是否一致的测试方法。
CI/CD	持续集成与持续交付，用自动化构建、测试和发布降低交付风险。
蓝绿发布	在两套环境之间切换流量以降低发布风险的策略。
灰度发布	逐步扩大用户范围观察新版本表现的发布策略。
回滚	发布失败或风险过高时恢复到上一稳定版本的操作。
可观测性	通过日志、指标、追踪理解系统运行状态和故障原因的能力。
故障复盘	对故障过程、影响、根因和改进措施进行结构化总结的活动。

附录 B 期末复习题索引

第 1 章
高价值复习题：为什么“功能能跑”不代表系统工程上已经完成？
题型：简答题
难度：中等

第 1 章
高价值复习题：用实验室预约系统说明 FR、QA、C 的区别。
题型：简答题
难度：中等

第 2 章
高价值复习题：在需求不确定和技术风险较高时，应如何选择开发节奏？
题型：案例分析
难度：拔高

第 2 章
高价值复习题：比较瀑布、原型、增量、迭代、螺旋模型的适用条件。
题型：简答题
难度：中等

第 3 章
高价值复习题：给定项目延期案例，分析范围、风险、里程碑和沟通问题。
题型：案例分析
难度：拔高

第 3 章
高价值复习题：为一个课程项目设计 WBS 和风险登记表。
题型：项目应用
难度：中等

第 4 章
高价值复习题：把“预约更方便”改写为可验证需求。
题型：案例分析
难度：中等

第 4 章
高价值复习题：说明用例文本和 SRS 在需求工程中的作用。
题型：简答题
难度：中等

第 5 章
高价值复习题：判断给定问题应使用 C4、UML、ERD 还是 DFD。
题型：案例分析
难度：中等

第 5 章
高价值复习题：为什么建模图不是装饰？
题型：简答题
难度：基础

第 6 章
高价值复习题：为什么课程项目首版通常更适合模块化单体？
题型：简答题
难度：中等

第 6 章
高价值复习题：根据案例写一条 ADR，并说明放弃方案。
题型：项目应用
难度：拔高

第 7 章
高价值复习题：如何从架构边界判断系统是否可测试？
题型：案例分析
难度：拔高

第 7 章
高价值复习题：测试金字塔和质量门禁分别解决什么问题？
题型：简答题
难度：中等

第 8 章
高价值复习题：给定上线故障，设计回滚、观测和复盘方案。
题型：案例分析
难度：拔高

第 8 章
高价值复习题：CI/CD、蓝绿发布、灰度发布的区别是什么？
题型：名词解释
难度：基础

附录 C 课程项目产出模板索引

M1
提交材料：选题说明
对应章节：第 1 章
简短说明：说明项目背景、目标用户、核心问题和复杂性来源。

M1
提交材料：利益相关者与典型用户
对应章节：第 4 章
简短说明：说明谁使用系统、谁受影响、谁提供规则和约束。

M1
提交材料：核心用例与需求规格初稿
对应章节：第 4 章
简短说明：至少写清主要成功场景、异常流程和验收标准。

M2
提交材料：C4 上下文图和容器图
对应章节：第 5 章
简短说明：表达系统边界、外部依赖和主要容器职责。

M2
提交材料：用例图、流程图、核心类图/数据模型
对应章节：第 5 章
简短说明：表达用户目标、关键流程、领域对象和数据约束。

M2
提交材料：建模检查清单
对应章节：第 5 章
简短说明：检查图是否有边界、关系、术语一致性和后续用途。

M3
提交材料：架构风格说明
对应章节：第 6 章
简短说明：说明采用模块化单体、分层、六边形等风格的理由。

M3
提交材料：ADR 与放弃方案
对应章节：第 6 章
简短说明：至少记录首版架构风格、一致性策略、外部依赖处理。

M3
提交材料：技术债记录
对应章节：第 6 章
简短说明：说明哪些简化是有意接受，何时偿还，观察什么指标。

M4
提交材料：测试计划与质量门禁
对应章节：第 7 章
简短说明：说明单元、接口、集成、安全测试和发布前检查条件。

M4

提交材料：交付与运维说明

对应章节：第 8 章

简短说明：说明部署步骤、配置、版本、回滚、日志和监控方式。

M4

提交材料：项目答辩材料

对应章节：第 1-8 章

简短说明：把需求、建模、架构、测试和交付材料整合为完整说明。